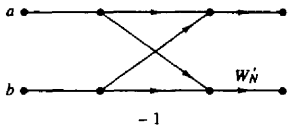


Figure 6.10 Basic butterfly computation in the decimation-in-frequency FFT algorithm.

stages of decimation, where each stage involves $N/2$ butterflies of the type shown in Fig. 6.10. Consequently, the computation of the N-point DFT via the decimation-in-frequency FFT algorithm, requires $(N/2)\log_2 N$ complex multiplications and $N\log_2 N$ complex additions, just as in the decimation-in-time algorithm. For illustrative purposes, the eight-point decimation-in-frequency algorithm is given in Fig. 6.11.

We observe from Fig. 6.11, that the input data $x(n)$ occurs in natural order, but the output DFT occurs in bit-reversed order. We also note that the computations are performed in place. However, it is possible to reconfigure the decimation-in-frequency algorithm so that the input sequence occurs in bit-reversed order while the output DFT occurs in normal order. Furthermore, if we abandon the requirement that the computations be done in place, it is also possible to have both the input data and the output DFT in normal order.

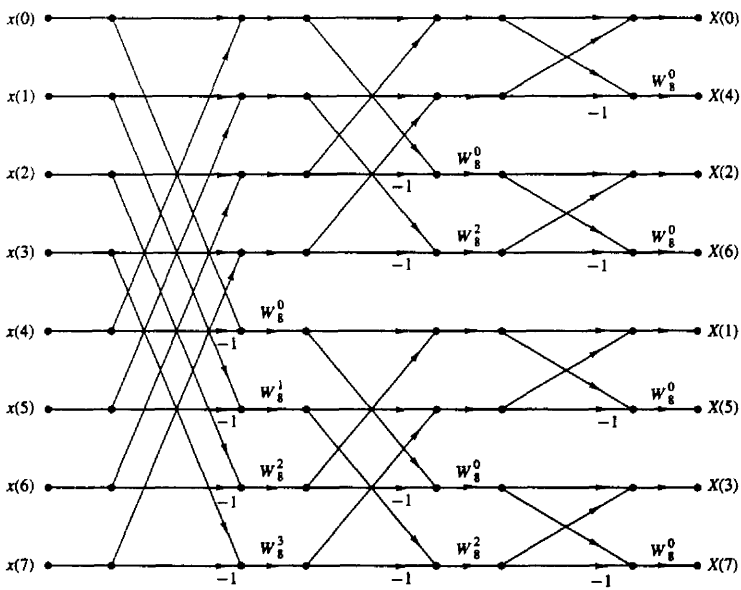


Figure 6.11 N = 8-point decimation-in-frequency FFT algorithmn.

6.1.4 Radix-4 FFT Algorithms

When the number of data points N in the DFT is a power of 4 (i.e., $N = 4^v$), we can, of course, always use a radix-2 algorithm for the computation. However, for this case, it is more efficient computationally to employ a radix-4 FFT algorithm.

Let us begin by describing a radix-4 decimation-in-time FFT algorithm, which is obtained by selecting $L = 4$ and $M = N/4$ in the divide-and-conquer approach described in Section 6.1.2. For this choice of L and M, we have l, $p = 0, 1, 2, 3$; m, $q = 0, 1, \ldots, N/4 - 1$; $n = 4m + l$; and $k = (N/4)p + q$. Thus we split or decimate the N-point input sequence into four subsequences, $x(4n)$, $x(4n + 1)$, $x(4n + 2)$, $x(4n + 3)$, $n = 0, 1, \ldots, N/4 - 1$.

By applying (6.1.15) we obtain

$$X(p, q) = \sum_{l=0}^{3} \left[W_N^{lq} F(l, q) \right] W_4^{lp} \qquad p = 0, 1, 2, 3 \tag{6.1.39}$$

where $F(l, q)$ is given by (6.1.16), that is,

$$F(l, q) = \sum_{m=0}^{(N/4)-1} x(l, m) W_{N/4}^{mq} \qquad \begin{matrix} l = 0, 1, 2, 3, \\ q = 0, 1, 2, \ldots, \dfrac{N}{4} - 1 \end{matrix} \tag{6.1.40}$$

and

$$x(l, m) = x(4m + l) \tag{6.1.41}$$

$$X(p, q) = X\left(\frac{N}{4}p + q\right) \tag{6.1.42}$$

Thus, the four $N/4$-point DFTs obtained from (6.1.40) are combined according to (6.1.39) to yield the N-point DFT. The expression in (6.1.39) for combining the $N/4$-point DFTs defines a radix-4 decimation-in-time butterfly, which can be expressed in matrix form as

$$\begin{bmatrix} X(0, q) \\ X(1, q) \\ X(2, q) \\ X(3, q) \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -j & -1 & j \\ 1 & -1 & 1 & -1 \\ 1 & j & -1 & -j \end{bmatrix} \begin{bmatrix} W_N^0 F(0, q) \\ W_N^q F(1, q) \\ W_N^{2q} F(2, q) \\ W_N^{3q} F(3, q) \end{bmatrix} \tag{6.1.43}$$

The radix-4 butterfly is depicted in Fig. 6.12(a) and in a more compact form in Fig. 6.12(b). Note that since $W_N^0 = 1$, each butterfly involves three complex multiplications, and 12 complex additions.

This decimation-in-time procedure can be repeated recursively v times. Hence the resulting FFT algorithm consists of v stages, where each stage contains $N/4$ butterflies. Consequently, the computational burden for the algorithm is $3vN/4 = (3N/8) \log_2 N$ complex multiplications and $(3N/2) \log_2 N$ complex additions. We note that the number of multiplications is reduced by 25%, but the number of additions has increased by 50% from $N \log_2 N$ to $(3N/2) \log_2 N$.

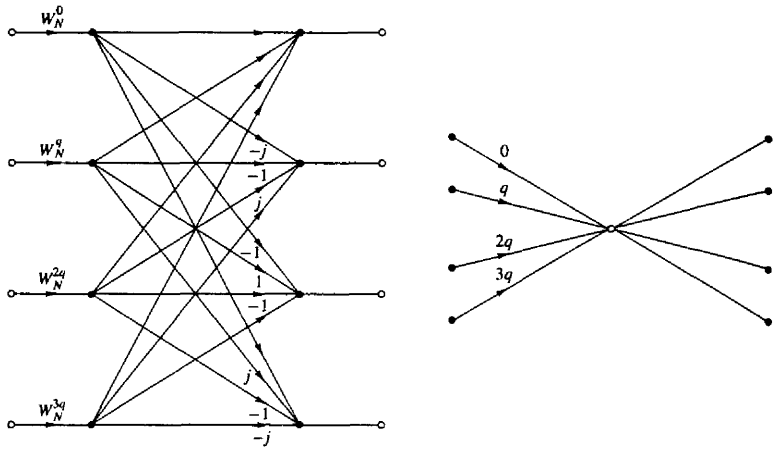


Figure 6.12 Basic butterfly computation in a radix-4 FFT algorithm.

It is interesting to note, however, that by performing the additions in two steps, it is possible to reduce the number of additions per butterfly from 12 to 8. This can be accomplished by expressing the matrix of the linear transformation in (6.1.43) as a product of two matrices as follows:

$$\begin{bmatrix} X(0,q) \\ X(1,q) \\ X(2,q) \\ X(3,q) \end{bmatrix} = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & -j \\ 1 & 0 & -1 & 0 \\ 0 & 1 & 0 & j \end{bmatrix} \begin{bmatrix} 1 & 0 & 1 & 0 \\ 1 & 0 & -1 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & -1 \end{bmatrix} \begin{bmatrix} W_N^0 F(0,q) \\ W_N^q F(1,q) \\ W_N^{2q} F(2,q) \\ W_N^{3q} F(3,q) \end{bmatrix} \tag{6.1.44}$$

Now each matrix multiplication involves four additions for a total of eight additions. Thus the total number of complex additions is reduced to $N \log_2 N$, which is identical to the radix-2 FFT algorithm. The computational savings results from the 25% reduction in the number of complex multiplications.

An illustration of a radix-4 decimation-in-time FFT algorithm is shown in Fig. 6.13 for $N = 16$. Note that in this algorithm, the input sequence is in normal order while the output DFT is shuffled. In the radix-4 FFT algorithm, where the decimation is by a factor of 4, the order of the decimated sequence can be determined by reversing the order of the number that represents the index n in a quaternary number system (i.e., the number system based on the digits 0, 1, 2, 3).

A radix-4 decimation-in-frequency FFT algorithm can be obtained by selecting $L = N/4$, $M = 4$; l, $p = 0, 1, \ldots, N/4 - 1$; m, $q = 0, 1, 2, 3$; $n = (N/4)m + l$; and $k = 4p + q$. With this choice of parameters, the general equation given by

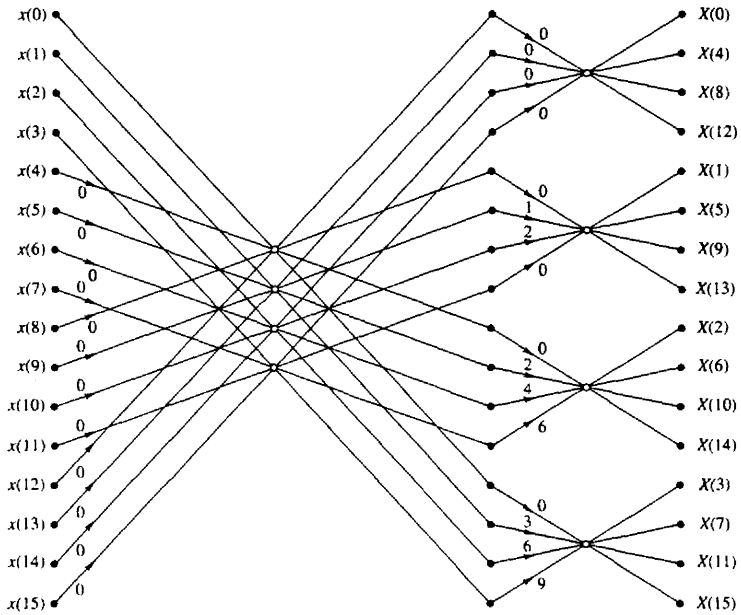


Figure 6.13 Sixteen-point radix-4 decimation-in-time algorithm with input in normal order and output in digit-reversed order.

(6.1.15) can be expressed as

$$X(p,q) = \sum_{l=0}^{(N/4)-1} G(l,q) W_{N/4}^{lp} \tag{6.1.45}$$

where

$$G(l,q) = W_N^{lq} F(l,q) \qquad \begin{matrix} q = 0,1,2,3 \\ l = 0,1,\ldots,\dfrac{N}{4}-1 \end{matrix} \tag{6.1.46}$$

and

$$F(l,q) = \sum_{m=0}^{3} x(l,m) W_4^{mq} \qquad \begin{matrix} q = 0,1,2,3 \\ l = 0,1,2,3,\ldots,\dfrac{N}{4}-1 \end{matrix} \tag{6.1.47}$$

We note that $X(p,q) = X(4p+q)$, $q = 0, 1, 2, 3$. Consequently, the N-point DFT is decimated into four $N/4$-point DFTs and hence we have a decimation-in-frequency FFT algorithm. The computations in (6.1.46) and (6.1.47) define

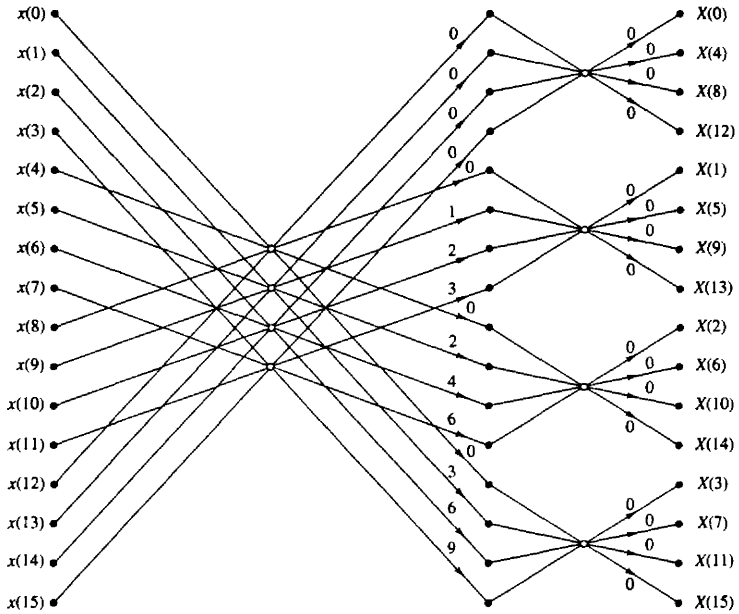


Figure 6.14 Sixteen-point, radix-4 decimation-in-frequency algorithm with input in normal order and output in digit-reversed order.

the basic radix-4 butterfly for the decimation-in-frequency algorithm. Note that the multiplications by the factors W_N^{lq} occur after the combination of the data points $x(l, m)$, just as in the case of the radix-2 decimation-in-frequency algorithm.

A 16-point radix-4 decimation-in-frequency FFT algorithm is shown in Fig. 6.14. Its input is in normal order and its output is in digit-reversed order. It has exactly the same computational complexity as the decimation-in-time radix-4 FFT algorithm.

For illustrative purposes, let us rederive the radix-4 decimation-in-frequency algorithm by breaking the N-point DFT formula into four smaller DFTs. We have

$$X(k) = \sum_{n=0}^{N-1} x(n) W_N^{kn}$$

$$= \sum_{n=0}^{N/4-1} x(n) W_N^{kn} + \sum_{n=N/4}^{N/2-1} x(n) W_N^{kn} + \sum_{n=N/2}^{3N/4-1} x(n) W_N^{kn} + \sum_{n=3N/4}^{N-1} x(n) W_N^{kn}$$

$$= \sum_{n=0}^{N/4-1} x(n)W_N^{kn} + W_N^{Nk/4} \sum_{n=0}^{N/4-1} x\left(n+\frac{N}{4}\right) W_N^{kn}$$
$$+ W_N^{kN/2} \sum_{n=0}^{N/4-1} x\left(n+\frac{N}{2}\right) W_N^{nk} + W_N^{3kN/4} \sum_{n=0}^{N/4-1} x\left(n+\frac{3N}{4}\right) W_N^{kn} \tag{6.1.48}$$

From the definition of the twiddle factors, we have

$$W_N^{kN/4} = (-j)^k \qquad W_N^{Nk/2} = (-1)^k \qquad W_N^{3Nk/4} = (j)^k \tag{6.1.49}$$

After substitution of (6.1.49) into (6.1.48), we obtain

$$X(k) = \sum_{n=0}^{N/4-1} \left[x(n) + (-j)^k x\left(n+\frac{N}{4}\right) + (-1)^k x\left(n+\frac{N}{4}\right) + (j)^k x\left(n+\frac{3N}{4}\right) \right] W_N^{nk} \tag{6.1.50}$$

The relation in (6.1.50) is not an $N/4$-point DFT because the twiddle factor depends on N and not on $N/4$. To convert it into an $N/4$-point DFT, we subdivide the DFT sequence into four $N/4$-point subsequences, $X(4k)$, $X(4k+1)$, $X(4k+2)$, and $X(4k+3)$, $k = 0, 1, \ldots, N/4-1$. Thus we obtain the radix-4 decimation-in-frequency DFT as

$$X(4k) = \sum_{n=0}^{N/4-1} \left[x(n) + x\left(n+\frac{N}{4}\right) + x\left(n+\frac{N}{2}\right) + x\left(n+\frac{3N}{4}\right) \right] W_N^0 W_{N/4}^{kn} \tag{6.1.51}$$

$$X(4k+1) = \sum_{n=0}^{N/4-1} \left[x(n) - jx\left(n+\frac{N}{4}\right) - x\left(n+\frac{N}{2}\right) + jx\left(n+\frac{3N}{4}\right) \right] W_N^n W_{N/4}^{kn} \tag{6.1.52}$$

$$X(4k+2) = \sum_{n=0}^{N/4-1} \left[x(n) - x\left(n+\frac{N}{4}\right) + x\left(n+\frac{N}{2}\right) - x\left(n+\frac{3N}{4}\right) \right] W_N^{2n} W_{N/4}^{kn} \tag{6.1.53}$$

$$X(4k+3) = \sum_{n=0}^{N/4-1} \left[x(n) + jx\left(n+\frac{N}{4}\right) - x\left(n+\frac{N}{2}\right) - jx\left(n+\frac{3N}{4}\right) \right] W_N^{3n} W_{N/4}^{kn} \tag{6.1.54}$$

where we have used the property $W_N^{4kn} = W_{N/4}^{kn}$. Note that the input to each $N/4$-point DFT is a linear combination of four signal samples scaled by a twiddle factor. This procedure is repeated ν times, where $\nu = \log_4 N$.

6.1.5 Split-Radix FFT Algorithms

An inspection of the radix-2 decimation-in-frequency flowgraph shown in Fig. 6.11 indicates that the even-numbered points of the DFT can be computed independently of the odd-numbered points. This suggests the possibility of using different computational methods for independent parts of the algorithm with the objective of reducing the number of computations. The split-radix FFT (SRFFT) algorithms exploit this idea by using both a radix-2 and a radix-4 decomposition in the same FFT algorithm.

We illustrate this approach with a decimation-in-frequency SRFFT algorithm due to Duhamel (1986). First, we recall that in the radix-2 decimation-in-frequency FFT algorithm, the even-numbered samples of the N-point DFT are given as

$$X(2k) = \sum_{n=0}^{N/2-1} \left[x(n) + x\left(n + \frac{N}{2}\right)\right] W_{N/2}^{nk} \qquad k = 0, 1, \ldots, \frac{N}{2} - 1 \tag{6.1.55}$$

Note that these DFT points can be obtained from an $N/2$-point DFT without any additional multiplications. Consequently, a radix-2 suffices for this computation.

The odd-numbered samples $\{X(2k+1)\}$ of the DFT require the premultiplication of the input sequence with the twiddle factors W_N^n. For these samples a radix-4 decomposition produces some computational efficiency because the four-point DFT has the largest multiplication-free butterfly. Indeed, it can be shown that using a radix greater than 4, does not result in a significant reduction in computational complexity.

If we use a radix-4 decimation-in-frequency FFT algorithm for the odd-numbered samples of the N-point DFT, we obtain the following $N/4$-point DFTs:

$$\begin{aligned} X(4k+1) = \sum_{n=0}^{N/4-1} &\{[x(n) - x(n+N/2)] \\ &- j[x(n+N/4) - x(n+3N/4)]\} W_N^n W_{N/4}^{kn} \end{aligned} \tag{6.1.56}$$

$$\begin{aligned} X(4k+3) = \sum_{n=0}^{N/4-1} &\{[x(n) - x(n+N/2)] \\ &+ j[x(n+N/4) - x(n+3N/4)]\} W_N^{3n} W_{N/4}^{kn} \end{aligned} \tag{6.1.57}$$

Thus the N-point DFT is decomposed into one $N/2$-point DFT without additional twiddle factors and two $N/4$-point DFTs with twiddle factors. The N-point DFT is obtained by successive use of these decompositions up to the last stage. Thus we obtain a decimation-in-frequency SRFFT algorithm.

Figure 6.15 shows the flow graph for an in-place 32-point decimation-in-frequency SRFFT algorithm. At stage A of the computation for $N = 32$, the

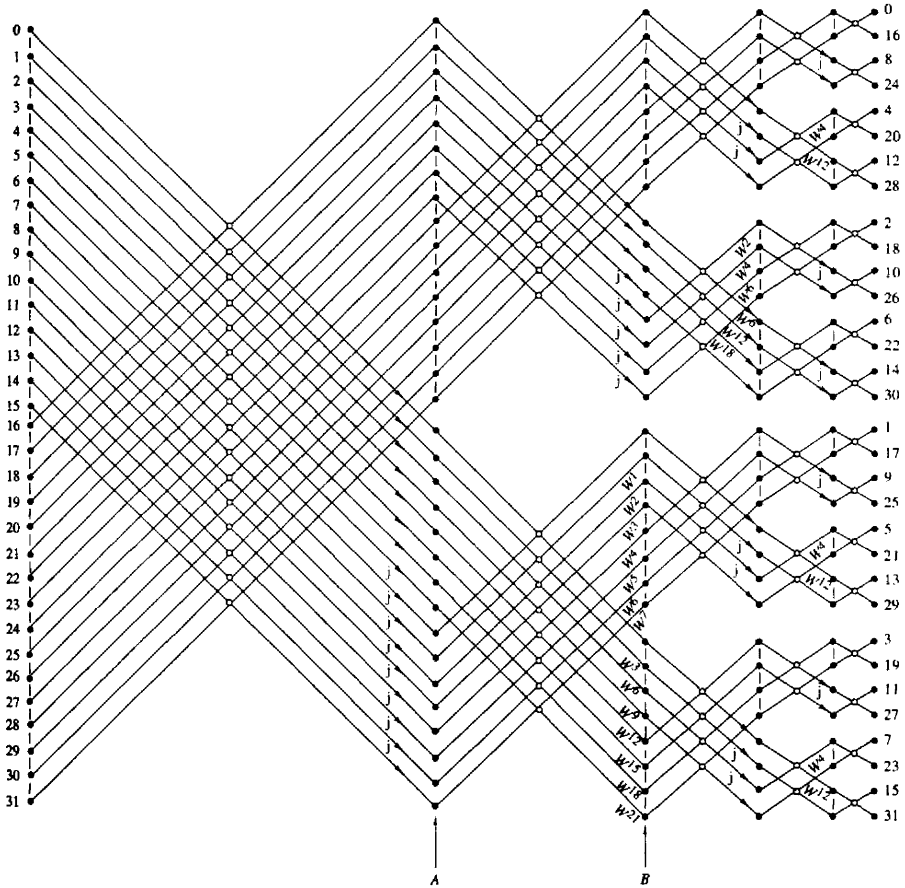


Figure 6.15 Length 32 split-radix FFT algorithms from paper by Duhamel (1986); reprinted with permission from the IEEE.

top 16 points constitute the sequence

$$g_0(n) = x(n) + x(n + N/2) \qquad 0 \le n \le 15 \tag{6.1.58}$$

This is the sequence required for the computation of $X(2k)$. The next 8 points constitute the sequence

$$g_1(n) = x(n) - x(n + N/2) \qquad 0 \le n \le 7 \tag{6.1.59}$$

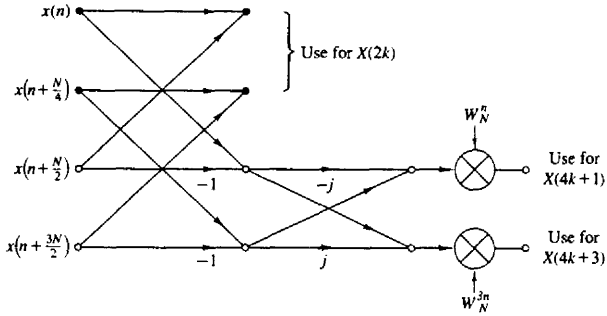


Figure 6.16 Butterfly for SRFFT algorithm.

The bottom eight points constitute the sequence $jg_2(n)$, where

$$g_2(n) = x(n + N/4) - x(n + 3N/4) \qquad 0 \le n \le 7 \tag{6.1.60}$$

The sequences $g_1(n)$ and $g_2(n)$ are used in the computation of $X(4k + 1)$ and $X(4k+3)$. Thus, at stage A we have completed the first decimation for the radix-2 component of the algorithm. At stage B, the bottom eight points constitute the computation of $[g_1(n)+jg_2(n)]W_{32}^{3n}$, $0 \le n \le 7$, which is used to compute $X(4k+3)$, $0 \le k \le 7$. The next eight points from the bottom constitute the computation of $[g_1(n) - jg_2(n)]\ W_{32}^{n}$, $0 \le n \le 7$, which is used to compute $X(4k + 1)$, $0 \le k \le 7$. Thus at stage B, we have completed the first decimation for the radix-4 algorithm, which results in two 8-point sequences. Hence the basic butterfly computation for the SRFFT algorithm has the "L-shaped" form illustrated in Fig. 6.16.

Now we repeat the steps in the computation above. Beginning with the top 16 points at stage A, we repeat the decomposition for the 16-point DFT. In other words, we decompose the computation into an eight-point, radix-2 DFT and two four-point, radix-4 DFTs. Thus at stage B, the top eight points constitute the sequence (with $N = 16$)

$$g_0'(n) = g_0(n) + g_0(n + N/2) \qquad 0 \le n \le 7 \tag{6.1.61}$$

and the next eight points constitute the two four-point sequences $g_1'(n)$ and $jg_2'(n)$, where

$$\begin{aligned} g_1'(n) &= g_0(n) - g_0(n + N/2) & 0 \le n \le 3 \\ g_2'(n) &= g_0(n + N/4) - g_0(n + 3N/4) & 0 \le n \le 3 \end{aligned} \tag{6.1.62}$$

The bottom 16 points of stage B are in the form of two eight-point DFTs. Hence each eight-point DFT is decomposed into a four-point, radix-2 DFT and a four-point, radix-4 DFT. In the final stage, the computations involve the combination of two-point sequences.

Table 6.2 presents a comparison of the number of *nontrivial* real multiplications and additions required to perform an N-point DFT with complex-valued

TABLE 6.2 NUMBER OF NONTRIVIAL REAL MULTIPLICATIONS AND ADDITIONS TO COMPUTE AN N-POINT COMPLEX DFT

	Real Multiplications				Real Additions			
N	Radix 2	Radix 4	Radix 8	Split Radix	Radix 2	Radix 4	Radix 8	Split Radix
16	24	20		20	152	148		148
32	88			68	408			388
64	264	208	204	196	1,032	976	972	964
128	712			516	2,504			2,308
256	1,800	1,392		1,284	5,896	5,488		5,380
512	4,360		3,204	3,076	13,566		12,420	12,292
1,024	10,248	7,856		7,172	30,728	28,336		27,652

Source: Extracted from Duhamel (1986).

data, using a radix-2, radix-4, radix-8, and a split-radix FFT. Note that the SRFFT algorithm requires the lowest number of multiplication and additions. For this reason, it is preferable in many practical applications.

Another type of SRFFT algorithm has been developed by Price (1990). Its relation to Duhamel's algorithm described previously can be seen by noting that the radix-4 DFT terms $X(4k+1)$ and $X(4k+3)$ involve the $N/4$-point DFTs of the sequences $[g_1(n) - jg_2(n)]W_N^n$ and $[g_1(n) + jg_2(n)]W_N^{3n}$, respectively. In effect, the sequences $g_1(n)$ and $g_2(n)$ are multiplied by the factor (vector) $(1, -j) = (1, W_{32}^8)$ and by W_N^n for the computation of $X(4k+1)$, while the computation of $X(4k+3)$ involves the factor $(1, j) = (1, W_{32}^{-8})$ and W_N^{3n}. Instead, one can rearrange the computation so that the factor for $X(4k+3)$ is $(-j, -1) = -(W_{32}^{-8}, 1)$. As a result of this phase rotation, the twiddle factors in the computation of $X(4k+3)$ become exactly the same as those for $X(4k+1)$, except that they occur in mirror image order. For example, at stage B of Fig. 6.15, the twiddle factors W^{2l}, $W^{18}, \ldots, W^3$ are replaced by $W^1, W^2, \ldots, W^7$, respectively. This mirror-image symmetry occurs at every subsequent stage of the algorithm. As a consequence, the number of twiddle factors that must be computed and stored is reduced by a factor of 2 in comparison to Duhamel's algorithm. The resulting algorithm is called the "mirror" FFT (MFFT) algorithm.

An additional factor-of-2 savings in storage of twiddle factors can be obtained by introducing a 90° phase offset at the midpoint of each twiddle array, which can be removed if necessary at the output of the SRFFT computation. The incorporation of this improvement into the SRFFT (or the MFFT) results in another algorithm, also due to Price (1990), called the "phase" FFT (PFFT) algorithm.

6.1.6 Implementation of FFT Algorithms

Now that we have described the basic radix-2 and radix-4 FFT algorithms, let us consider some of the implementation issues. Our remarks apply directly to

radix-2 algorithms, although similar comments may be made about radix-4 and higher-radix algorithms.

Basically, the radix-2 FFT algorithm consists of taking two data points at a time from memory, performing the butterfly computations and returning the resulting numbers to memory. This procedure is repeated many times ($(N \log_2 N)/2$ times) in the computation of an N-point DFT.

The butterfly computations require the twiddle factors $\{W_N^k\}$ at various stages in either natural or bit-reversed order. In an efficient implementation of the algorithm, the phase factors are computed once and stored in a table, either in normal order or in bit-reversed order, depending on the specific implementation of the algorithm.

Memory requirement is another factor that must be considered. If the computations are performed in place, the number of memory locations required is $2N$ since the numbers are complex. However, we can instead double the memory to 4N, thus simplifying the indexing and control operations in the FFT algorithms. In this case we simply alternate in the use of the two sets of memory locations from one stage of the FFT algorithm to the other. Doubling of the memory also allows us to have both the input sequence and the output sequence in normal order.

There are a number of other implementation issues regarding indexing, bit reversal, and the degree of parallelism in the computations. To a large extent, these issues are a function of the specific algorithm and the type of implementation, namely, a hardware or software implementation. In implementations based on a fixed-point arithmetic, or floating-point arithmetic on small machines, there is also the issue of round-off errors in the computation. This topic is considered in Section 6.4.

Although the FFT algorithms described previously were presented in the context of computing the DFT efficiently, they can also be used to compute the IDFT, which is

$$x(n) = \frac{1}{N} \sum_{k=0}^{N-1} X(k) W_N^{-nk} \tag{6.1.63}$$

The only difference between the two transforms is the normalization factor $1/N$ and the sign of the phase factor W_N. Consequently, an FFT algorithm for computing the DFT, can be converted to an FFT algorithm for computing the IDFT by changing the sign on all the phase factors and dividing the final output of the algorithm by N.

In fact, if we take the decimation-in-time algorithm that we described in Section 6.1.3, reverse the direction of the flow graph, change the sign on the phase factors, interchange the output and input, and finally, divide the output by N, we obtain a decimation-in-frequency FFT algorithm for computing the IDFT. On the other hand, if we begin with the decimation-in-frequency FFT algorithm described in Section 6.1.3 and repeat the changes described above, we obtain a decimation-in-time FFT algorithm for computing the IDFT. Thus it is a simple matter to devise FFT algorithms for computing the IDFT.

Finally, we note that the emphasis in our discussion of FFT algorithms was on radix-2, radix-4, and split-radix algorithms. These are by far the most widely used in practice. When the number of data points is not a power of 2 or 4, it is a simple matter to pad the sequence $x(n)$ with zeros such that $N = 2^v$ or $N = 4^v$.

The measure of complexity for FFT algorithms that we have emphasized is the required number of arithmetic operations (multiplications and additions). Although this is a very important benchmark for computational complexity, there are other issues to be considered in practical implementation of FFT algorithms. These include the architecture of the processor, the available instruction set, the data structures for storing twiddle factors, and other considerations.

For general-purpose computers, where the cost of the numerical operations dominate, radix-2, radix-4, and split-radix FFT algorithms are good candidates. However, in the case of special-purpose digital signal processors, featuring single-cycle multiply-and-accumulate operation, bit-reversed addressing, and a high degree of instruction parallelism, the structural regularity of the algorithm is equally important as arithmetic complexity. Hence for DSP processors, radix-2 or radix-4 decimation-in-frequency FFT algorithms are preferable in terms of speed and accuracy. The irregular structure of the SRFFT may render it less suitable for implementation on digital signal processors. Structural regularity is also important in the implementation of FFT algorithms on vector processors, multiprocessors, and in VLSI. Interprocessor communication is an important consideration in such implementations on parallel processors.

In conclusion, we have presented several important considerations in the implementation of FFT algorithms. Advances in digital signal processing technology, in hardware and software, will continue to influence the choice among FFT algorithms for various practical applications.

6.2 APPLICATIONS OF FFT ALGORITHMS

The FFT algorithms described in the preceding section find application in a variety of areas, including linear filtering, correlation, and spectrum analysis. Basically, the FFT algorithm is used as an efficient means to compute the DFT and the IDFT.

In this section we consider the use of the FFT algorithm in linear filtering and in the computation of the crosscorrelation of two sequences. The use of the FFT in spectrum analysis is considered in Chapter 12. In addition we illustrate how to enhance the efficiency of the FFT algorithm by forming complex-valued sequences from real-valued sequences prior to the computation of the DFT.

6.2.1 Efficient Computation of the DFT of Two Real Sequences

The FFT algorithm is designed to perform complex multiplications and additions, even though the input data may be real valued. The basic reason for this situation is

that the phase factors are complex and hence, after the first stage of the algorithm, all variables are basically complex-valued.

In view of the fact that the algorithm can handle complex-valued input sequences, we can exploit this capability in the computation of the DFT of two real-valued sequences.

Suppose that $x_1(n)$ and $x_2(n)$ are two real-valued sequences of length N, and let $x(n)$ be a complex-valued sequence defined as

$$x(n) = x_1(n) + jx_2(n) \qquad 0 \le n \le N-1 \tag{6.2.1}$$

The DFT operation is linear and hence the DFT of $x(n)$ can be expressed as

$$X(k) = X_1(k) + jX_2(k) \tag{6.2.2}$$

The sequences $x_1(n)$ and $x_2(n)$ can be expressed in terms of $x(n)$ as follows:

$$x_1(n) = \frac{x(n) + x^*(n)}{2} \tag{6.2.3}$$

$$x_2(n) = \frac{x(n) - x^*(n)}{2j} \tag{6.2.4}$$

Hence the DFTs of $x_1(n)$ and $x_2(n)$ are

$$X_1(k) = \frac{1}{2}\{DFT[x(n)] + DFT[x^*(n)]\} \tag{6.2.5}$$

$$X_2(k) = \frac{1}{2j}\{DFT[x(n)] - DFT[x^*(n)]\} \tag{6.2.6}$$

Recall that the DFT of $x^*(n)$ is $X^*(N-k)$. Therefore,

$$X_1(k) = \frac{1}{2}[X(k) + X^*(N-k)] \tag{6.2.7}$$

$$X_2(k) = \frac{1}{j2}[X(k) - X^*(N-k)] \tag{6.2.8}$$

Thus, by performing a single DFT on the complex-valued sequence $x(n)$, we have obtained the DFT of the two real sequences with only a small amount of additional computation that is involved in computing $X_1(k)$ and $X_2(k)$ from $X(k)$ by use of (6.2.7) and (6.2.8).

6.2.2 Efficient Computation of the DFT of a 2*N*-Point Real Sequence

Suppose that $g(n)$ is a real-valued sequence of $2N$ points. We now demonstrate how to obtain the $2N$-point DFT of $g(n)$ from computation of one N-point DFT involving complex-valued data. First, we define

$$\begin{aligned} x_1(n) &= g(2n) \\ x_2(n) &= g(2n+1) \end{aligned} \tag{6.2.9}$$

Thus we have subdivided the $2N$-point real sequence into two N-point real sequences. Now we can apply the method described in the preceding section.

Let $x(n)$ be the N-point complex-valued sequence

$$x(n) = x_1(n) + jx_2(n) \tag{6.2.10}$$

From the results of the preceding section, we have

$$\begin{aligned} X_1(k) &= \frac{1}{2}[X(k) + X^*(N-k)] \\ X_2(k) &= \frac{1}{2j}[X(k) - X^*(N-k)] \end{aligned} \tag{6.2.11}$$

Finally, we must express the $2N$-point DFT in terms of the two N-point DFTs, $X_1(k)$ and $X_2(k)$. To accomplish this, we proceed as in the decimation-in-time FFT algorithm, namely,

$$\begin{aligned} G(k) &= \sum_{n=0}^{N-1} g(2n) W_{2N}^{2nk} + \sum_{n=0}^{N-1} g(2n+1) W_{2N}^{(2n+1)k} \\ &= \sum_{n=0}^{N-1} x_1(n) W_N^{nk} + W_{2N}^{k} \sum_{n=0}^{N-1} x_2(n) W_N^{nk} \end{aligned}$$

Consequently,

$$\begin{aligned} G(k) &= X_1(k) + W_2^k N X_2(k) \qquad k = 0, 1, \ldots, N-1 \\ G(k+N) &= X_1(k) - W_2^k N X_2(k) \qquad k = 0, 1, \ldots, N-1 \end{aligned} \tag{6.2.12}$$

Thus we have computed the DFT of a $2N$-point real sequence from one N-point DFT and some additional computation as indicated by (6.2.11) and (6.2.12).

6.2.3 Use of the FFT Algorithm in Linear Filtering and Correlation

An important application of the FFT algorithm is in FIR linear filtering of long data sequences. In Chapter 5 we described two methods, the overlap-add and the overlap-save methods for filtering a long data sequence with an FIR filter, based on the use of the DFT. In this section we consider the use of these two methods in conjunction with the FFT algorithm for computing the DFT and the IDFT.

Let $h(n)$, $0 \le n \le M-1$, be the unit sample response of the FIR filter and let $x(n)$ denote the input data sequence. The block size of the FFT algorithm is N, where $N = L + M - 1$ and L is the number of new data samples being processed by the filter. We assume that for any given value of M, the number L of data samples is selected so that N is a power of 2. For purposes of this discussion, we consider only radix-2 FFT algorithms.

The N-point DFT of $h(n)$, which is padded by $L-1$ zeros, is denoted as $H(k)$. This computation is performed once via the FFT and the resulting N complex numbers are stored. To be specific we assume that the decimation-in-frequency

FFT algorithm is used to compute $H(k)$. This yields $H(k)$ in bit-reversed order, which is the way it is stored in memory.

In the overlap-save method, the first $M-1$ data points of each data block are the last $M-1$ data points of the previous data block. Each data block contains L new data points, such that $N = L + M - 1$. The N-point DFT of each data block is performed by the FFT algorithm. If the decimation-in-frequency algorithm is employed, the input data block requires no shuffling and the values of the DFT occur in bit-reversed order. Since this is exactly the order of $H(k)$, we can multiply the DFT of the data, say $X_m(k)$, with $H(k)$ and thus the result

$$Y_m(k) = H(k)X_m(k)$$

is also in bit-reversed order.

The inverse DFT (IDFT) can be computed by use of an FFT algorithm that takes the input in bit-reversed order and produces an output in normal order. Thus there is no need to shuffle any block of data either in computing the DFT or the IDFT.

If the overlap-add method is used to perform the linear filtering, the computational method using the FFT algorithm is basically the same. The only difference is that the N-point data blocks consist of L new data points and $M-1$ additional zeros. After the IDFT is computed for each data block, the N-point filtered blocks are overlapped as indicated in Section 5.3.2, and the $M-1$ overlapping data points between successive output records are added together.

Let us assess the computational complexity of the FFT method for linear filtering. For this purpose, the one-time computation of $H(k)$ is insignificant and can be ignored. Each FFT requires $(N/2)\log_2 N$ complex multiplications and $N \log_2 N$ additions. Since the FFT is performed twice, once for the DFT and once for the IDFT, the computational burden is $N \log_2 N$ complex multiplications and $2N \log_2 N$ additions. There are also N complex multiplications and $N-1$ additions required to compute $Y_m(k)$. Therefore, we have $(N \log_2 2N)/L$ complex multiplications per output data point and approximately $(2N \log_2 2N)/L$ additions per output data point. The overlap-add method requires an incremental increase of $(M-1)/L$ in the number of additions.

By way of comparison, a direct form realization of the FIR filter involves M real multiplications per output point if the filter is not linear phase, and $M/2$ if it is linear phase (symmetric). Also, the number of additions is $M-1$ per output point (see Sec. 8.2).

It is interesting to compare the efficiency of the FFT algorithm with the direct form realization of the FIR filter. Let us focus on the number of multiplications, which are more time consuming than additions. Suppose that $M = 128 = 2^7$ and $N = 2^\nu$. Then the number of complex multiplications per output point for an FFT size of $N = 2^\nu$ is

$$c(\nu) = \frac{N \log_2 2N}{L} = \frac{2^\nu(\nu+1)}{N-M+1}$$

$$\approx \frac{2^\nu(\nu+1)}{2^\nu - 2^7}$$

TABLE 6.3 COMPUTATIONAL COMPLEXITY

Size of FFT $\nu = \log_2 N$	$c(\nu)$ Number of Complex Multiplications per Output Point
9	13.3
10	12.6
11	12.8
12	13.4
14	15.1

The values of $c(\nu)$ for different values of ν are given in Table 6.3. We observe that there is an optimum value of ν which minimizes $c(\nu)$. For the FIR filter of size $M = 128$, the optimum occurs at $\nu = 10$.

We should emphasize that $c(\nu)$ represents the number of complex multiplications for the FFT-based method. The number of real multiplications is four times this number. However, even if the FIR filter has linear phase (see Sec. 8.2), the number of computations per output point is still less with the FFT-based method. Furthermore, the efficiency of the FFT method can be improved by computing the DFT of two successive data blocks simultaneously, according to the method just described. Consequently, the FFT-based method is indeed superior from a computational point of view when the filter length is relatively large.

The computation of the cross correlation between two sequences by means of the FFT algorithm is similar to the linear FIR filtering problem just described. In practical applications involving crosscorrelation, at least one of the sequences has finite duration and is akin to the impulse response of the FIR filter. The second sequence may be a long sequence which contains the desired sequence corrupted by additive noise. Hence the second sequence is akin to the input to the FIR filter. By time reversing the first sequence and computing its DFT, we have reduced the cross correlation to an equivalent convolution problem (i.e., a linear FIR filtering problem). Therefore, the methodology we developed for linear FIR filtering by use of the FFT applies directly.

6.3 A LINEAR FILTERING APPROACH TO COMPUTATION OF THE DFT

The FFT algorithm takes N points of input data and produces an output sequence of N points corresponding to the DFT of the input data. As we have shown, the radix-2 FFT algorithm performs the computation of the DFT in $(N/2)\log_2 N$ multiplications and $N \log_2 N$ additions for an N-point sequence.

There are some applications where only a selected number of values of the DFT are desired, but the entire DFT is not required. In such a case, the FFT algorithm may no longer be more efficient than a direct computation of the desired values of the DFT. In fact, when the desired number of values of

the DFT is less than $\log_2 N$, a direct computation of the desired values is more efficient.

The direct computation of the DFT can be formulated as a linear filtering operation on the input data sequence. As we will demonstrate, the linear filter takes the form of a parallel bank of resonators where each resonator selects one of the frequencies $\omega_k = 2\pi k/N$, $k = 0, 1, \ldots, N-1$, corresponding to the N frequencies in the DFT.

There are other applications in which we require the evaluation of the z-transform of a finite-duration sequence at points other than the unit circle. If the set of desired points in the z-plane possesses some regularity, it is possible to also express the computation of the z-transform as a linear filtering operation. In this connection, we introduce another algorithm, called the chirp-z transform algorithm, which is suitable for evaluating the z-transform of a set of data on a variety of contours in the z-plane. This algorithm is also formulated as a linear filtering of a set of input data. As a consequence, the FFT algorithm can be used to compute the chirp-z transform and thus to evaluate the z-transform at various contours in the z-plane, including the unit circle.

6.3.1 The Goertzel Algorithm

The Goertzel algorithm exploits the periodicity of the phase factors $\{W_N^k\}$ and allows us to express the computation of the DFT as a linear filtering operation. Since $W_N^{-kN} = 1$, we can multiply the DFT by this factor. Thus

$$X(k) = W_N^{-kN} \sum_{m=0}^{N-1} x(m) W_N^{km} = \sum_{m=0}^{N-1} x(m) W_N^{-k(N-m)} \tag{6.3.1}$$

We note that (6.3.1) is in the form of a convolution. Indeed, if we define the sequence $y_k(n)$ as

$$y_k(n) = \sum_{m=0}^{N-1} x(m) W_N^{-k(n-m)} \tag{6.3.2}$$

then it is clear that $y_k(n)$ is the convolution of the finite-duration input sequence $x(n)$ of length N with a filter that has an impulse response

$$h_k(n) = W_N^{-kn} u(n) \tag{6.3.3}$$

The output of this filter at $n = N$ yields the value of the DFT at the frequency $\omega_k = 2\pi k/N$. That is,

$$X(k) = y_k(n)|_{n=N} \tag{6.3.4}$$

as can be verified by comparing (6.3.1) with (6.3.2).

The filter with impulse response $h_k(n)$ has the system function

$$H_k(z) = \frac{1}{1 - W_N^{-k} z^{-1}} \tag{6.3.5}$$

This filter has a pole on the unit circle at the frequency $\omega_k = 2\pi k/N$. Thus, the entire DFT can be computed by passing the block of input data into a parallel bank of N single-pole filters (resonators), where each filter has a pole at the corresponding frequency of the DFT.

Instead of performing the computation of the DFT as in (6.3.2), via convolution, we can use the difference equation corresponding to the filter given by (6.3.5) to compute $y_k(n)$ recursively. Thus we have

$$y_k(n) = W_N^{-k} y_k(n-1) + x(n) \qquad y_k(-1) = 0 \tag{6.3.6}$$

The desired output is $X(k) = y_k(N)$, for $k = 0, 1, \ldots, N-1$. To perform this computation, we can compute once and store the phase factors W_N^{-k}.

The complex multiplications and additions inherent in (6.3.6) can be avoided by combining the pairs of resonators possessing complex-conjugate poles. This leads to two-pole filters with system functions of the form

$$H_k(z) = \frac{1 - W_N^k z^{-1}}{1 - 2\cos(2\pi k/N)z^{-1} + z^{-2}} \tag{6.3.7}$$

The direct form II realization of the system illustrated in Fig. 6.17 is described by the difference equation

$$v_k(n) = 2\cos\frac{2\pi k}{N} v_k(n-1) - v_k(n-2) + x(n) \tag{6.3.8}$$

$$y_k(n) = v_k(n) - W_N^k v_k(n-1) \tag{6.3.9}$$

with initial conditions $v_k(-1) = v_k(-2) = 0$.

The recursive relation in (6.3.8) is iterated for $n = 0, 1, \ldots, N$, but the equation in (6.3.9) is computed only once at time $n = N$. Each iteration requires one real multiplication and two additions. Consequently, for a real input sequence $x(n)$, this algorithm requires $N + 1$ real multiplications to yield not only $X(k)$ but also, due to symmetry, the value of $X(N-k)$.

The Goertzel algorithm is particularly attractive when the DFT is to be computed at a relatively small number M of values, where $M \leq \log_2 N$. Otherwise, the FFT algorithm is a more efficient method.

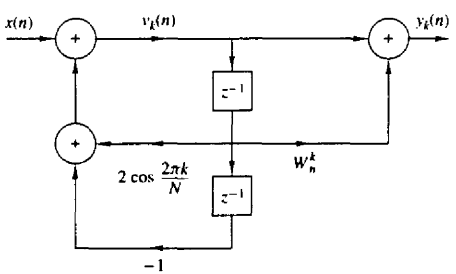


Figure 6.17 Direct form II realization of two-pole resonator for computing the DFT.

6.3.2 The Chirp-z Transform Algorithm

The DFT of an N-point data sequence $x(n)$ has been viewed as the z-transform of $x(n)$ evaluated at N equally spaced points on the unit circle. It has also been viewed as N equally spaced samples of the Fourier transform of the data sequence $x(n)$. In this section we consider the evaluation of $X(z)$ on other contours in the z-plane, including the unit circle.

Suppose that we wish to compute the values of the z-transform of $x(n)$ at a set of points $\{z_k\}$. Then,

$$X(z_k) = \sum_{n=0}^{N-1} x(n) z_k^{-n} \qquad k = 0, 1, \ldots, L-1 \tag{6.3.10}$$

For example, if the contour is a circle of radius r and the z_k are N equally spaced points, then

$$\begin{aligned} z_k &= re^{j2\pi kn/N} \qquad k = 0, 1, 2, \ldots, N-1 \\ X(z_k) &= \sum_{n=0}^{N-1} [x(n)r^{-n}] e^{-j2\pi kn/N} \qquad k = 0, 1, 2, \ldots, N-1 \end{aligned} \tag{6.3.11}$$

In this case the FFT algorithm can be applied on the modified sequence $x(n)r^{-n}$.

More generally, suppose that the points z_k in the z-plane fall on an arc which begins at some point

$$z_0 = r_0 e^{j\theta_0}$$

and spirals either in toward the origin or out away from the origin such that the points $\{z_k\}$ are defined as

$$z_k = r_0 e^{j\theta_0} (R_0 e^{j\phi_0})^k \qquad k = 0, 1, \ldots, L-1 \tag{6.3.12}$$

Note that if $R_0 < 1$, the points fall on a contour that spirals toward the origin and if $R_0 > 1$, the contour spirals away from the origin. If $R_0 = 1$, the contour is a circular arc of radius r_0. If $r_0 = 1$ and $R_0 = 1$, the contour is an arc of the unit circle. The latter contour would allow us to compute the frequency content of the sequence $x(n)$ at a dense set of L frequencies in the range covered by the arc without having to compute a large DFT, that is, a DFT of the sequence $x(n)$ padded with many zeros to obtain the desired resolution in frequency. Finally, if $r_0 = R_0 = 1$, $\theta_0 = 0$, $\phi_0 = 2\pi/N$, and $L = N$, the contour is the entire unit circle and the frequencies are those of the DFT. The various contours are illustrated in Fig. 6.18.

When points $\{z_k\}$ in (6.3.12) are substituted into the expression for the z-transform, we obtain

$$\begin{aligned} X(z_k) &= \sum_{n=0}^{N-1} x(n) z_k^{-n} \\ &= \sum_{n=0}^{N-1} x(n) (r_0 e^{j\theta_0})^{-n} V^{-nk} \end{aligned} \tag{6.3.13}$$

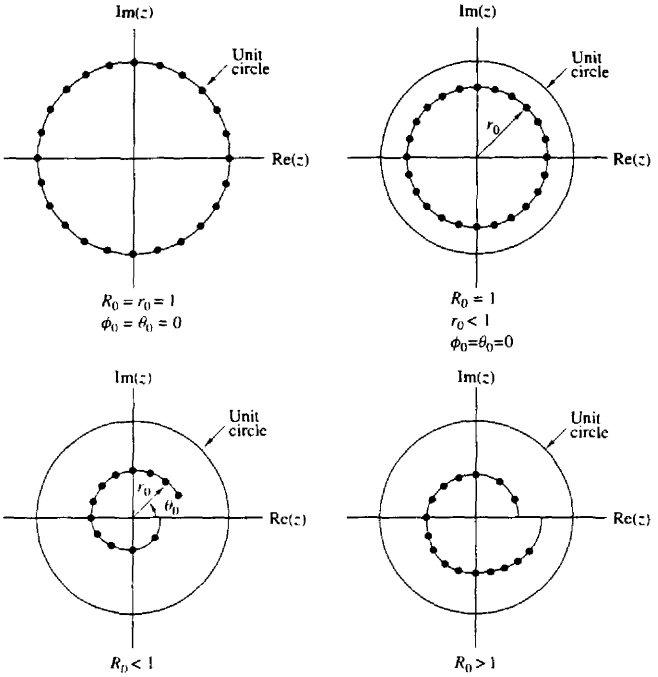


Figure 6.18 Some examples of contours on which we may evaluate the z-transform.

where, by definition,

$$V = R_0 e^{j\phi_0} \tag{6.3.14}$$

We can express (6.3.13) in the form of a convolution, by noting that

$$nk = \tfrac{1}{2}[n^2 + k^2 - (k-n)^2] \tag{6.3.15}$$

Substitution of (6.3.15) into (6.3.13) yields

$$X(z_k) = V^{-k^2/2} \sum_{n=0}^{N-1} [x(n)(r_0 e^{j\theta_0})^{-n} V^{-n^2/2}] V^{(k-n)^2/2} \tag{6.3.16}$$

Let us define a new sequence $g(n)$ as

$$g(n) = x(n)(r_0 e^{j\theta_0})^{-n} V^{-n^2/2} \tag{6.3.17}$$

Then (6.3.16) can be expressed as

$$X(z_k) = V^{-k^2/2} \sum_{n=0}^{N-1} g(n) V^{(k-n)^2/2} \tag{6.3.18}$$

The summation in (6.3.18) can be interpreted as the convolution of the sequence $g(n)$ with the impulse response $h(n)$ of a filter, where

$$h(n) = V^{n^2/2} \tag{6.3.19}$$

Consequently, (6.3.18) may be expressed as

$$\begin{aligned} X(z_k) &= V^{-k^2/2} y(k) \\ &= \frac{y(k)}{h(k)} \qquad k = 0, 1, \ldots, L-1 \end{aligned} \tag{6.3.20}$$

where $y(k)$ is the output of the filter

$$y(k) = \sum_{n=0}^{N-1} g(n)h(k-n) \qquad k = 0, 1, \ldots, L-1 \tag{6.3.21}$$

We observe that both $h(n)$ and $g(n)$ are complex-valued sequences.

The sequence $h(n)$ with $R_0 = 1$ has the form of a complex exponential with argument $\omega n = n^2\phi_0/2 = (n\phi_0/2)n$. The quantity $n\phi_0/2$ represents the frequency of the complex exponential signal, which increases linearly with time. Such signals are used in radar systems and are called *chirp signals*. Hence the z-transform evaluated as in (6.3.18) is called the *chirp-z transform*.

The linear convolution in (6.3.21) is most efficiently done by use of the FFT algorithm. The sequence $g(n)$ is of length N. However, $h(n)$ has infinite duration. Fortunately, only a portion $h(n)$ is required to compute the L values of $X(z)$.

Since we will compute the convolution in (6.3.1) via the FFT, let us consider the circular convolution of the N-point sequence $g(n)$ with an M-point section of $h(n)$, where $M > N$. In such a case, we know that the first $N-1$ points contain aliasing and that the remaining $M - N + 1$ points are identical to the result that would be obtained from a linear convolution of $h(n)$ with $g(n)$. In view of this, we should select a DFT of size

$$M = L + N - 1$$

which would yield L valid points and $N-1$ points corrupted by aliasing.

The section of $h(n)$ that is needed for this computation corresponds to the values of $h(n)$ for $-(N-1) \le n \le (L-1)$, which is of length $M = L + N - 1$, as observed from (6.3.21). Let us define the sequence $h_1(n)$ of length M as

$$h_1(n) = h(n - N + 1) \qquad n = 0, 1, \ldots, M-1 \tag{6.3.22}$$

and compute its M-point DFT via the FFT algorithm to obtain $H_1(k)$. From $x(n)$ we compute $g(n)$ as specified by (6.3.17), pad $g(n)$ with $L-1$ zeros, and compute its M-point DFT to yield $G(k)$. The IDFT of the product $Y_1(k) = G(k)H_1(k)$ yields the M-point sequence $y_1(n)$, $n = 0, 1, \ldots, M-1$. The first $N-1$ points of $y_1(n)$ are corrupted by aliasing and are discarded. The desired values are $y_1(n)$ for $N-1 \le n \le M-1$, which correspond to the range $0 \le n \le L-1$ in (6.3.21),

that is,

$$y(n) = y_1(n + N - 1) \qquad n = 0, 1, \ldots, L - 1 \tag{6.3.23}$$

Alternatively, we can define a sequence $h_2(n)$ as

$$h_2(n) = \begin{cases} h(n), & 0 \le n \le L - 1 \\ h(n - N - L + 1), & L \le n \le M - 1 \end{cases} \tag{6.3.24}$$

The M-point DFT of $h_2(n)$ yields $H_2(k)$, which when multiplied by $G(k)$ yields $Y_2(k) = G(k)H_2(k)$. The IDFT of $Y_2(k)$ yields the sequence $y_2(n)$ for $0 \le n \le M-1$. Now the desired values of $y_2(n)$ are in the range $0 \le n \le L - 1$, that is,

$$y(n) = y_2(n) \qquad n = 0, 1, \ldots, L - 1 \tag{6.3.25}$$

Finally, the complex values $X(z_k)$ are computed by dividing $y(k)$ by $h(k)$, $k = 0, 1, \ldots, L - 1$, as specified by (6.3.20).

In general, the computational complexity of the chirp-z transform algorithm described above is of the order of $M \log_2 M$ complex multiplications, where $M = N + L - 1$. This number should be compared with the product, $N \cdot L$, the number of computations required by direct evaluation of the z-transform. Clearly, if L is small, direct computation is more efficient. However, if L is large, then the chirp-z transform algorithm is more efficient.

The chirp-z transform method has been implemented in hardware to compute the DFT of signals. For the computation of the DFT, we select $r_0 = R_0 = 1$, $\theta_0 = 0$, $\phi_0 = 2\pi/N$, and $L = N$. In this case

$$\begin{aligned} V^{-n^2/2} &= e^{-j\pi n^2/N} \\ &= \cos \frac{\pi n^2}{N} - j \sin \frac{\pi n^2}{N} \end{aligned} \tag{6.3.26}$$

The chirp filter with impulse response

$$\begin{aligned} h(n) &= V^{n^2/2} \\ &= \cos \frac{\pi n^2}{N} + j \sin \frac{\pi n^2}{N} \\ &= h_r(n) + j h_i(n) \end{aligned} \tag{6.3.27}$$

has been implemented as a pair of FIR filters with coefficients $h_r(n)$ and $h_i(n)$, respectively. Both *surface acoustic wave* (SAW) devices and *charge coupled devices* (CCD) have been used in practice for the FIR filters. The cosine and sine sequences given in (6.3.26) needed for the premultiplications and postmultiplications are usually stored in a read-only memory (ROM). Furthermore, we note that if only the magnitude of the DFT is desired, the postmultiplications are unnecessary. In this case,

$$|X(z_k)| = |y(k)| \qquad k = 0, 1, \ldots, N - 1 \tag{6.3.28}$$

as illustrated in Fig. 6.19. Thus the linear FIR filtering approach using the chirp-z transform has been implemented for the computation of the DFT.

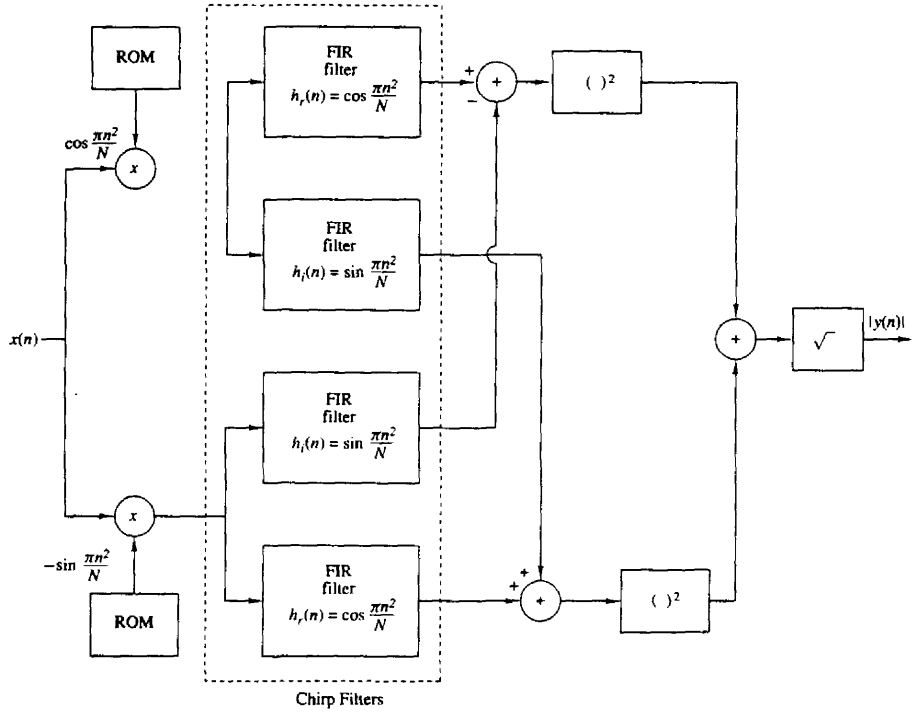


Figure 6.19 Block diagram illustrating the implementation of the chirp-z transform for computing the DFT (magnitude only).

6.4 QUANTIZATION EFFECTS IN THE COMPUTATION OF THE DFT*

As we have observed in our previous discussions, the DFT plays an important role in many digital signal processing applications, including FIR filtering, the computation of the correlation between signals, and spectral analysis. For this reason it is important for us to know the effect of quantization errors in its computation. In particular, we shall consider the effect of round-off errors due to the multiplications performed in the DFT with fixed-point arithmetic.

The model that we shall adopt for characterizing round-off errors in multiplication is the additive white noise model that we use in the statistical analysis of round-off errors in IIR and FIR filters (see Fig. 7.34). Although the statistical

*It is recommended that the reader review Section 7.5 prior to reading this section.

analysis is performed for rounding, the analysis can be easily modified to apply to truncation in two's-complement arithmetic (see Sec. 7.5.3).

Of particular interest is the analysis of round-off errors in the computation of the DFT via the FFT algorithm. However, we shall first establish a benchmark by determining the round-off errors in the direct computation of the DFT.

6.4.1 Quantization Errors in the Direct Computation of the DFT

Given a finite-duration sequence $\{x(n)\}$, $0 \le n \le N-1$, the DFT of $\{x(n)\}$ is defined as

$$X(k) = \sum_{n=0}^{N-1} x(n) W_N^{kn} \qquad k = 0, 1, \ldots, N-1 \tag{6.4.1}$$

where $W_N = e^{-j2\pi/N}$. We assume that in general, $\{x(n)\}$ is a complex-valued sequence. We also assume that the real and imaginary components of $\{x(n)\}$ and $\{W_N^{kn}\}$ are represented by b bits. Consequently, the computation of the product $x(n)W_N^{kn}$ requires four real multiplications. Each real multiplication is rounded from $2b$ bits to b bits, and hence there are four quantization errors for each complex-valued multiplication.

In the direct computation of the DFT, there are N complex-valued multiplications for each point in the DFT. Therefore, the total number of real multiplications in the computation of a single point in the DFT is $4N$. Consequently, there are $4N$ quantization errors.

Let us evaluate the variance of the quantization errors in a fixed-point computation of the DFT. First, we make the following assumptions about the statistical properties of the quantization errors.

1. The quantization errors due to rounding are uniformly distributed random variables in the range $(-\Delta/2, \Delta/2)$ where $\Delta = 2^{-b}$.
2. The $4N$ quantization errors are mutually uncorrelated.
3. The $4N$ quantization errors are uncorrelated with the sequence $\{x(n)\}$.

Since each of the quantization errors has a variance

$$\sigma_e^2 = \frac{\Delta^2}{12} = \frac{2^{-2b}}{12} \tag{6.4.2}$$

the variance of the quantization errors from the $4N$ multiplications is

$$\begin{aligned} \sigma_q^2 &= 4N\sigma_e^2 \\ &= \frac{N}{3} \cdot 2^{-2b} \end{aligned} \tag{6.4.3}$$

Hence the variance of the quantization error is proportional to the size of DFT. Note that when N is a power of 2 (i.e., $N = 2^v$), the variance can be expressed

as

$$\sigma_q^2 = \frac{2^{-2(b-v/2)}}{3} \tag{6.4.4}$$

This expression implies that every fourfold increase in the size N of the DFT requires an additional bit in computational precision to offset the additional quantization errors.

To prevent overflow, the input sequence to the DFT requires scaling. Clearly, an upper bound on $|X(k)|$ is

$$|X(k)| \leq \sum_{n=0}^{N-1} |x(n)| \tag{6.4.5}$$

If the dynamic range in addition is $(-1, 1)$, then $|X(k)| < 1$ requires that

$$\sum_{n=0}^{N-1} |x(n)| < 1 \tag{6.4.6}$$

If $|x(n)|$ is initially scaled such that $|x(n)| < 1$ for all n, then each point in the sequence can be divided by N to ensure that (6.4.6) is satisfied.

The scaling implied by (6.4.6) is extremely severe. For example, suppose that the signal sequence $\{x(n)\}$ is white and, after scaling, each value $|x(n)|$ of the sequence is uniformly distributed in the range $(-1/N, 1/N)$. Then the variance of the signal sequence is

$$\sigma_x^2 = \frac{(2/N)^2}{12} = \frac{1}{3N^2} \tag{6.4.7}$$

and the variance of the output DFT coefficients $|X(k)|$ is

$$\begin{aligned} \sigma_X^2 &= N\sigma_x^2 \\ &= \frac{1}{3N} \end{aligned} \tag{6.4.8}$$

Thus the signal-to-noise power ratio is

$$\frac{\sigma_X^2}{\sigma_q^2} = \frac{2^{2b}}{N^2} \tag{6.4.9}$$

We observe that the scaling is responsible for reducing the SNR by N and the combination of scaling and quantization errors result in a total reduction that is proportional to N^2. Hence scaling the input sequence $\{x(n)\}$ to satisfy (6.4.6) imposes a severe penalty on the signal-to-noise ratio in the DFT.

Example 6.4.1

Use (6.4.9) to determine the number of bits required to compute the DFT of a 1024-point sequence with a SNR of 30 dB.

Solution The size of the sequence is $N = 2^{10}$. Hence the SNR is

$$10 \log_{10} \frac{\sigma_X^2}{\sigma_q^2} = 10 \log_{10} 2^{2b-20}$$

For an SNR of 30 dB, we have

$$3(2b - 20) = 30$$

$$b = 15 \text{ bits}$$

Note that the 15 bits is the precision for both multiplication and addition.

Instead of scaling the input sequence $\{x(n)\}$, suppose we simply require that $|x(n)| < 1$. Then we must provide a sufficiently large dynamic range for addition such that $|X(k)| < N$. In such a case, the variance of the sequence $\{|x(n)|\}$ is $\sigma_x^2 = \frac{1}{3}$, and hence the variance of $|X(k)|$ is

$$\sigma_X^2 = N\sigma_x^2 = \frac{N}{3} \tag{6.4.10}$$

Consequently, the SNR is

$$\frac{\sigma_X^2}{\sigma_q^2} = 2^{2b} \tag{6.4.11}$$

If we repeat the computation in Example 6.4.1, we find that the number of bits required to achieve a SNR of 30 dB is $b = 5$ bits. However, we need an additional 10 bits for the accumulator (the adder) to accommodate the increase in the dynamic range for addition. Although we did not achieve any reduction in the dynamic range for addition, we have managed to reduce the precision in multiplication from 15 bits to 5 bits, which is highly significant.

6.4.2 Quantization Errors in FFT Algorithms

As we have shown, the FFT algorithms require significantly fewer multiplications than the direct computation of the DFT. In view of this we might conclude that the computation of the DFT via an FFT algorithm will result in smaller quantization errors. Unfortunately, that is not the case, as we will demonstrate.

Let us consider the use of fixed-point arithmetic in the computation of a radix-2 FFT algorithm. To be specific, we select the radix-2, decimation-in-time algorithm illustrated in Fig. 6.20 for the case $N = 8$. The results on quantization errors that we obtain for this radix-2 FFT algorithm are typical of the results obtained with other radix-2 and higher radix algorithms.

We observe that each butterfly computation involves one complex-valued multiplication or, equivalently, four real multiplications. We ignore the fact that some butterflies contain a trivial multiplication by ± 1. If we consider the butterflies that affect the computation of any one value of the DFT, we find that, in general, there are $N/2$ in the first stage of the FFT, $N/4$ in the second stage, $N/8$ in the third state, and so on, until the last stage, where there is only one. Consequently, the number of butterflies per output point is

$$\begin{aligned} 2^{\nu-1} + 2^{\nu-2} + \cdots + 2 + 1 &= 2^{\nu-1}\left[1 + \left(\tfrac{1}{2}\right) + \cdots + \left(\tfrac{1}{2}\right)^{\nu-1}\right] \\ &= 2^{\nu}[1 - \left(\tfrac{1}{2}\right)^{\nu}] = N - 1 \end{aligned} \tag{6.4.12}$$

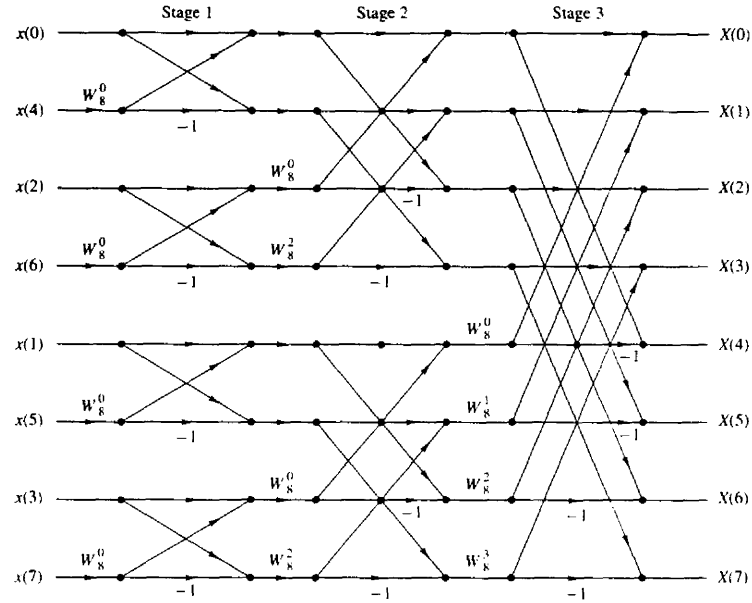


Figure 6.20 Decimation-in-time FFT algorithm.

For example, the butterflies that affect the computation of $X(3)$ in the eight-point FFT algorithm of Fig. 6.20 are illustrated in Fig. 6.21.

The quantization errors introduced in each butterfly propagate to the output. Note that the quantization errors introduced in the first stage propagate through $(\nu - 1)$ stages, those introduced in the second stage propagate through $(\nu - 2)$ stages, and so on. As these quantization errors propagate through a number of subsequent stages, they are phase shifted (phase rotated) by the phase factors W_N^{kn}. These phase rotations do not change the statistical properties of the quantization errors and, in particular, the variance of each quantization error remains invariant.

If we assume that the quantization errors in each butterfly are uncorrelated with the errors in other butterflies, then there are $4(N - 1)$ errors that affect the output of each point of the FFT. Consequently, the variance of the total quantization error at the output is

$$\sigma_q^2 = 4(N-1)\frac{\Delta^2}{12} \approx \frac{N\Delta^2}{3} \tag{6.4.13}$$

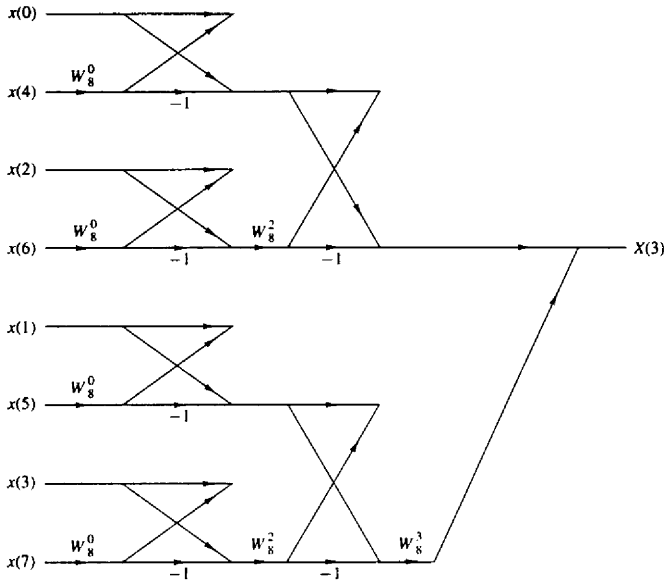


Figure 6.21 Butterflies that affect the computation of $X(3)$.

where $\Delta = 2^{-b}$. Hence

$$\sigma_q^2 = \frac{N}{3} \cdot 2^{-2b} \tag{6.4.14}$$

This is exactly the same result that we obtained for the direct computation of the DFT.

The result in (6.4.14) should not be surprising. In fact, the FFT algorithm does not reduce the number of multiplications required to compute a single point of the DFT. It does, however, exploit the periodicities in W_N^{kn} and thus reduces the number of multiplications in the computation of the entire block of N points in the DFT.

As in the case of the direct computation of the DFT, we must scale the input sequence to prevent overflow. Recall that if $|x(n)| < 1/N$, $0 \leq n \leq N - 1$, then $|X(k)| < 1$ for $0 \leq k \leq N - 1$. Thus overflow is avoided. With this scaling, the relations in (6.4.7), (6.4.8), and (6.4.9), obtained previously for the direct computation of the DFT, apply to the FFT algorithm as well. Consequently, the same SNR is obtained for the FFT.

Since the FFT algorithm consists of a sequence of stages, where each stage contains butterflies that involve pairs of points, it is possible to devise a different scaling strategy that is not as severe as dividing each input point by N. This

alternative scaling strategy is motivated by the observation that the intermediate values $|X_n(k)|$ in the $n = 1, 2, \ldots, \nu$ stages of the FFT algorithm satisfy the conditions (see Problem 6.35)

$$\begin{aligned} \max[|X_{n+1}(k)|, |X_{n+1}(l)|] &\geq \max[|X_n(k)|, |X_n(l)|] \\ \max[|X_{n+1}(k)|, |X_{n+1}(l)|] &\leq 2\max[|X_n(k)|, |X_n(l)|] \end{aligned} \tag{6.4.15}$$

In view of these relations, we can distribute the total scaling of $1/N$ into each of the stages of the FFT algorithm. In particular, if $|x(n)| < 1$, we apply a scale factor of $\frac{1}{2}$ in the first stage so that $|x(n)| < \frac{1}{2}$. Then the output of each subsequent stage in the FFT algorithm is scaled by $\frac{1}{2}$, so that after ν stages we have achieved an overall scale factor of $(\frac{1}{2})^\nu = 1/N$. Thus overflow in the computation of the DFT is avoided.

This scaling procedure does not affect the signal level at the output of the FFT algorithm, but it significantly reduces the variance of the quantization errors at the output. Specifically, each factor of $\frac{1}{2}$ reduces the variance of a quantization error term by a factor of $\frac{1}{4}$. Thus the $4(N/2)$ quantization errors introduced in the first stage are reduced in variance by $(\frac{1}{4})^{\nu-1}$, the $4(N/4)$ quantization errors introduced in the second stage are reduced in variance by $(\frac{1}{4})^{\nu-2}$, and so on. Consequently, the total variance of the quantization errors at the output of the FFT algorithm is

$$\begin{aligned} \sigma_q^2 &= \frac{\Delta^2}{12}\left\{4\left(\frac{N}{2}\right)\left(\frac{1}{4}\right)^{\nu-1} + 4\left(\frac{N}{4}\right)\left(\frac{1}{4}\right)^{\nu-2} + 4\left(\frac{N}{8}\right)\left(\frac{1}{4}\right)^{\nu-3} + \cdots + 4\right\} \\ &= \frac{\Delta^2}{3}\left\{\left(\frac{1}{2}\right)^{\nu-1} + \left(\frac{1}{2}\right)^{\nu-2} + \cdots + \frac{1}{2} + 1\right\} \\ &= \frac{2\Delta^2}{3}\left[1 - \left(\frac{1}{2}\right)^{\nu}\right] \approx \frac{2}{3}\cdot 2^{-2b} \end{aligned} \tag{6.4.16}$$

where the factor $(\frac{1}{2})^\nu$ is negligible.

We now observe that (6.4.16) is no longer proportional to N. On the other hand, the signal has the variance $\sigma_X^2 = 1/3N$, as given in (6.4.8). Hence the SNR is

$$\begin{aligned} \frac{\sigma_X^2}{\sigma_q^2} &= \frac{1}{2N}\cdot 2^{2b} \\ &= 2^{2b-\nu-1} \end{aligned} \tag{6.4.17}$$

Thus, by distributing the scaling of $1/N$ uniformly throughout the FFT algorithm, we have achieved an SNR that is inversely proportional to N instead of N^2.

Example 6.4.2

Determine the number of bits required to compute an FFT of 1024 points with an SNR of 30 dB when the scaling is distributed as described above.

Solution The size of the FFT is $N = 2^{10}$. Hence the SNR according to (6.4.17) is

$$\begin{aligned} 10\log_{10} 2^{2b-\nu-1} &= 30 \\ 3(2b - 11) &= 30 \\ b &= \frac{21}{2}(11 \text{ bits}) \end{aligned}$$

This can be compared with the 15 bits required if all the scaling is performed in the first stage of the FFT algorithm.

6.5 SUMMARY AND REFERENCES

The focus of this chapter was on the efficient computation of the DFT. We demonstrated that by taking advantage of the symmetry and periodicity properties of the exponential factors W_N^{kn}, we can reduce the number of complex multiplications needed to compute the DFT from N^2 to $N \log_2 N$ when N is a power of 2. As we indicated, any sequence can be augmented with zeros, such that $N = 2^\nu$.

For decades, FFT-type algorithms were of interest to mathematicians who were concerned with computing values of Fourier series by hand. However, it was not until Cooley and Tukey (1965) published their well-known paper that the impact and significance of the efficient computation of the DFT was recognized. Since then the Cooley–Tukey FFT algorithm and its various forms, for example, the algorithms of Singleton (1967, 1969), have had a tremendous influence on the use of the DFT in convolution, correlation, and spectrum analysis. For a historical perspective on the FFT algorithm, the reader is referred to the paper by Cooley et al. (1967).

The split-radix FFT (SRFFT) algorithm described in Section 9.3.5 is due to Duhamel and Hollmann (1984, 1986). The "mirror" FFT (MFFT) and "phase" FFT (PFFT) algorithms were described to the authors by R. Price. The exploitation of symmetry properties in the data to reduce the computation time are described in a paper by Swarztrauber (1986).

Over the years, a number of tutorial papers have been published on FFT algorithms. We cite the early papers by Brigham and Morrow (1967), Cochran et al. (1967), Bergland (1969), and Cooley et al. (1967, 1969).

The recognition that the DFT can be arranged and computed as a linear convolution is also highly significant. Goertzel (1968) indicated that the DFT can be computed via linear filtering, although the computational savings of this approach is rather modest, as we have observed. More significant is the work of Bluestein (1970), who demonstrated that the computation of the DFT can be formulated as a chirp linear filtering operation. This work led to the development of the chirp-z transform algorithm by Rabiner et al. (1969).

In addition to the FFT algorithms described in this chapter, there are other efficient algorithms for computing the DFT, some of which further reduce the

number of multiplications, but usually require more additions. Of particular importance is an algorithm due to Rader and Brenner (1976), the class of prime factor algorithms, such as the Good algorithm (1971), and the Winograd algorithm (1976, 1978). For a description of these and related algorithms, the reader may refer to the text by Blahut (1985).

PROBLEMS

6.1 Show that each of the numbers

$$e^{j(2\pi/N)k} \qquad 0 \le k \le N-1$$

corresponds to an Nth root of unity. Plot these numbers as phasors in the complex plane and illustrate, by means of this figure, the orthogonality property

$$\sum_{n=0}^{N-1} e^{j(2\pi/N)kn} e^{-j(2\pi/N)ln} = \begin{cases} N, & \text{if } k = l \\ 0, & \text{if } k \neq l \end{cases}$$

6.2 **(a)** Show that the phase factors can be computed recursively by

$$W_N^{ql} = W_N^{q} W_N^{q(l-1)}$$

(b) Perform this computation once using single-precision floating-point arithmetic and once using only four significant digits. Note the deterioration due to the accumulation of round-off errors in the later case.

(c) Show how the results in part (b) can be improved by resetting the result to the correct value $-j$, each time $ql = N/4$.

6.3 Let $x(n)$ be a real-valued N-point ($N = 2^v$) sequence. Develop a method to compute an N-point DFT $X'(k)$, which contains only the odd harmonics [i.e., $X'(k) = 0$ if k is even] by using only a real $N/2$-spoint DFT.

6.4 A designer has available a number of eight-point FFT chips. Show explicitly how he should interconnect three such chips in order to compute a 24-point DFT.

6.5 The z-transform of the sequence $x(n) = u(n) - u(n-7)$ is sampled at five points on the unit circle as follows

$$x(k) = X(z)|_{z} = e^{j2\pi k/5} \qquad k = 0, 1, 2, 3, 4$$

Determine the inverse DFT $x'(n)$ of $X(k)$. Compare it with $x(n)$ and explain the results.

6.6 Consider a finite-duration sequence $x(n)$, $0 \le n \le 7$, with z-transform $X(z)$. We wish to compute $X(z)$ at the following set of values:

$$z_k = 0.8e^{j[(2\pi k/8)+(\pi/8)]} \qquad 0 \le k \le 7$$

(a) Sketch the points $\{z_k\}$ in the complex plane.

(b) Determine a sequence $s(n)$ such that its DFT provides the desired samples of $X(z)$.

6.7 Derive the radix-2 decimation-in-time FFT algorithm given by (6.1.26) and (6.1.27) as a special case of the more general algorithmic procedure given by (6.1.16) through (6.1.18).

6.8 Compute the eight-point DFT of the sequence

$$x(n) = \begin{cases} 1, & 0 \le n \le 7 \\ 0, & \text{otherwise} \end{cases}$$

by using the decimation-in-frequency FFT algorithm described in the text.

6.9 Derive the signal flow graph for the $N = 16$ point, radix-4 decimation-in-time FFT algorithm in which the input sequence is in normal order and the computations are done in place.

6.10 Derive the signal flow graph for the $N = 16$ point, radix-4 decimation-in-frequency FFT algorithm in which the input sequence is in digit-reversed order and the output DFT is in normal order.

6.11 Compute the eight-point DFT of the sequence

$$x(n) = \left\{\frac{1}{2}, \frac{1}{2}, \frac{1}{2}, \frac{1}{2}, 0, 0, 0, 0\right\}$$

using the in-place radix-2 decimation-in-time and radix-2 decimation-in-frequency algorithms. Follow exactly the corresponding signal flow graphs and keep track of all the intermediate quantities by putting them on the diagrams.

6.12 Compute the 16-point DFT of the sequence

$$x(n) = \cos\frac{\pi}{2}n \qquad 0 \le n \le 15$$

using the radix-4 decimation-in-time algorithm.

6.13 Consider the eight-point decimation-in-time (DIT) flow graph in Fig. 6.6.

(a) What is the gain of the "signal path" that goes from $x(7)$ to $X(2)$?

(b) How many paths lead from the input to a given output sample? Is this true for every output sample?

(c) Compute $X(3)$ using the operations dictated by this flow graph.

6.14 Draw the flow graph for the decimation-in-frequency (DIF) SRFFT algorithm for $N = 16$. What is the number of nontrivial multiplications?

6.15 Derive the algorithm and draw the $N = 8$ flow graph for the DIT SRFFT algorithm. Compare your flow graph with the DIF radix-2 FFT flow graph shown in Fig. 6.11.

6.16 Show that the product of two complex numbers $(a+jb)$ and $(c+jd)$ can be performed with three real multiplications and five additions using the algorithm

$$x_R = (a-b)d + (c-d)a$$

$$x_I = (a-b)d + (c+d)b$$

where

$$x = x_R + jx_I = (a+jb)(c+jd)$$

6.17 Explain how the DFT can be used to compute N equispaced samples of the z-transform, of an N-point sequence, on a circle of radius r.

6.18 A real-valued N-point sequence $x(n)$ is called DFT bandlimited if its DFT $X(k) = 0$ for $k_0 \le k \le N - k_0$. We insert $(L-1)N$ zeros in the middle of $X(k)$ to obtain the following LN-point DFT

$$X'(k) = \begin{cases} X(k), & 0 \le k \le k_0 - 1 \\ 0, & k_0 \le k \le LN - k_0 \\ X(k + N - LN), & LN - k_0 + 1 \le k \le LN - 1 \end{cases}$$

Show that

$$Lx'(Ln) = x(n) \qquad 0 \le n \le N - 1$$

where

$$x'(n) \overset{\substack{DFT \\ LN}}{\longleftrightarrow} X'(k)$$

Explain the meaning of this type of processing by working out an example with $N = 4$, $L = 1$, and $X(k) = \{1, 0, 0, 1\}$.

6.19 Let $X(k)$ be the N-point DFT of the sequence $x(n)$, $0 \le n \le N - 1$. What is the N-point DFT of the sequence $s(n) = X(n)$, $0 \le n \le N - 1$?

6.20 Let $X(k)$ be the N-point DFT of the sequence $x(n)$, $0 \le n \le N - 1$. We define a $2N$-point sequence $y(n)$ as

$$y(n) = \begin{cases} x\left(\frac{n}{2}\right), & n \text{ even} \\ 0, & n \text{ odd} \end{cases}$$

Express the $2N$-point DFT of $y(n)$ in terms of $X(k)$.

6.21 **(a)** Determine the z-transform $W(z)$ of the Hanning window $w(n) = \left(1 - \cos \frac{2\pi n}{N-1}\right)/2$.

(b) Determine a formula to compute the N-point DFT $X_w(k)$ of the signal $x_w(n) = w(n)x(n)$, $0 \le n \le N - 1$, from the N-point DFT $X(k)$ of the signal $x(n)$.

6.22 Create a DFT coefficient table that uses only $N/4$ memory locations to store the first quadrant of the sine sequence (assume N even).

6.23 Determine the computational burden of the algorithm given by (6.2.12) and compare it with the computational burden required in the $2N$-point DFT of $g(n)$. Assume that the FFT algorithm is a radix-2 algorithm.

6.24 Consider an IIR system described by the difference equation

$$y(n) = -\sum_{k=1}^{N} a_k y(n-k) + \sum_{k=0}^{M} b_k x(n-k)$$

Describe a procedure that computes the frequency response $H\left(\frac{2\pi}{N}k\right)$, $k = 0, 1, \ldots, N - 1$ using the FFT algorithm ($N = 2^\nu$).

6.25 Develop a radix-3 decimation-in-time FFT algorithm for $N = 3^\nu$ and draw the corresponding flow graph for $N = 9$. What is the number of required complex multiplications? Can the operations be performed in place?

6.26 Repeat Problem 6.25 for the DIF case.

6.27 *FFT input and output pruning* In many applications we wish to compute only a few points M of the N-point DFT of a finite-duration sequence of length L (i.e., $M << N$ and $L << N$).

(a) Draw the flow graph of the radix-2 DIF FFT algorithm for $N = 16$ and eliminate [i.e., prune] all signal paths that originate from zero inputs assuming that only $x(0)$ and $x(1)$ are nonzero.

(b) Repeat part (a) for the radix-2 DIT algorithm.

(c) Which algorithm is better if we wish to compute all points of the DFT? What happens if we want to compute only the points $X(0)$, $X(1)$, $X(2)$, and $X(3)$? Establish a rule to choose between DIT and DIF pruning depending on the values of M and L.

(d) Give an estimate of saving in computations in terms of M, L, and N.

6.28 *Parallel computation of the DFT* Suppose that we wish to compute an $N = 2^p 2^\nu$ point DFT using 2^p digital signal processors (DSPs). For simplicity we assume that $p = \nu = 2$. In this case each DSP carries out all the computations that are necessary to compute 2^ν DFT points.

(a) Using the radix-2 DIF flow graph, show that to avoid data shuffling, the entire sequence $x(n)$ should be loaded to the memory of each DSP.

(b) Identify and redraw the portion of the flow graph that is executed by the DSP that computes the DFT samples $X(2)$, $X(10)$, $X(6)$, and $X(14)$.

(c) Show that, if we use $M = 2^\nu$ DSPs, the computation speed-up S is given by

$$S = M \frac{\log_2 N}{\log_2 N - \log_2 M + 2(M-1)}$$

6.29 Develop an inverse radix-2 DIT FFT algorithm starting with the definition. Draw the flow graph for computation and compare with the corresponding flow graph for the direct FFT. Can the IFFT flow graph be obtained from the one for the direct FFT?

6.30 Repeat Problem 6.29 for the DIF case.

6.31 Show that an FFT on data with Hermitian symmetry can be derived by reversing the flow graph of an FFT for real data.

6.32 Determine the system function $H(z)$ and the difference equation for the system that uses the Goertzel algorithm to compute the DFT value $X(N-k)$.

6.33 (a) Suppose that $x(n)$ is a finite-duration sequence of $N = 1024$ points. It is desired to evaluate the z-transform $X(z)$ of the sequence at the points

$$z_k = e^{j(2\pi/1024)k} \qquad k = 0, 100, 200, \ldots, 1000$$

by using the most efficient method or algorithm possible. Describe an algorithm for performing this computation efficiently. Explain how you arrived at your answer by giving the various options or algorithms that can be used.

(b) Repeat part (a) if $X(z)$ is to be evaluated at

$$z_k = 2(0.9)^k e^{j[(2\pi/5000)k + \pi/2]} \qquad k = 0, 1, 2, \ldots, 999$$

6.34 Repeat the analysis for the variance of the quantization error, carried out in Section 6.4.2, for the decimation-in-frequency radix-2 FFT algorithm.

6.35 The basic butterfly in the radix-2 decimation-in-time FFT algorithm is

$$X_{n+1}(k) = X_n(k) + W_N^m X_n(l)$$

$$X_{n+1}(l) = X_n(k) - W_N^m X_n(l)$$

(a) If we require that $|X_n(k)| < \frac{1}{2}$ and $|X_n(l)| < \frac{1}{2}$, show that

$$|\operatorname{Re}[X_{n+1}(k)]| < 1, \qquad |Re[X_{n+1}(l)]| < 1$$
$$|\operatorname{Im}[X_{n+1}(k)]| < 1, \qquad |Im[X_{n+1}(l)]| < 1$$

Thus overflow does not occur.

(b) Prove that

$$\max[|X_{n+1}(k)|, |X_{n+1}(l)|] \geq \max[|X_n(k)|, |X_n(l)|]$$
$$\max[|X_{n+1}(k)|, |X_{n+1}(l)|] \leq 2 \max[|X_n(k)|, |X_n(l)|]$$

6.36* *Computation of the DFT* Use an FFT subroutine to compute the following DFTs and plot the magnitudes $|X(k)|$ of the DFTs.

(a) The 64-point DFT of the sequence

$$x(n) = \begin{cases} 1, & n = 0, 1, \ldots, 15 \quad (N_1 = 16) \\ 0, & \text{otherwise} \end{cases}$$

(b) The 64-point DFT of the sequence

$$x(n) = \begin{cases} 1, & n = 0, 1, \ldots, 7 \quad (N_1 = 8) \\ 0, & \text{otherwise} \end{cases}$$

(c) The 128-point DFT of the sequence in part (a).

(d) The 64-point DFT of the sequence

$$x(n) = \begin{cases} 10e^{j(\pi/8)n}, & n = 0, 1, \ldots, 63 \quad (N_1 = 64) \\ 0, & \text{otherwise} \end{cases}$$

Answer the following questions.

(1) What is the frequency interval between successive samples for the plots in parts (a), (b), (c), and (d)?

(2) What is the value of the spectrum at zero frequency (dc value) obtained from the plots in parts (a), (b), (c), (d)?
From the formula

$$X(k) = \sum_{n=0}^{N-1} x(n) e^{-j(2\pi/N)nk}$$

compute the theoretical values for the dc value and check these with the computer results.

(3) In plots (a), (b), and (c), what is the *frequency interval* between successive nulls in the spectrum? What is the relationship between N_1 of the sequence $x(n)$ and the frequency interval between successive nulls?

(4) Explain the difference between the plots obtained from parts (a) and (c).

6.37* *Identification of pole positions in a system* Consider the system described by the difference equation

$$y(n) = -r^2 y(n-2) + x(n)$$

(a) Let $r = 0.9$ and $x(n) = \delta(n)$. Generate the output sequence $y(n)$ for $0 \leq n \leq 127$. Compute the $N = 128$ point DFT $\{Y(k)\}$ and plot $\{|Y(k)|\}$.

(b) Compute the $N = 128$ point DFT of the sequence

$$w(n) = (0.92)^{-n} y(n)$$

where $y(n)$ is the sequence generated in part (a). Plot the DFT values $|W(k)|$. What can you conclude from the plots in parts (a) and (b)?

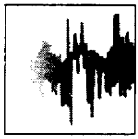
(c) Let $r = 0.5$ and repeat part (a).

(d) Repeat part (b) for the sequence

$$w(n) = (0.55)^{-n} y(n)$$

where $y(n)$ is the sequence generated in part (c). What can you conclude from the plots in parts (c) and (d)?

(e) Now let the sequence generated in part (c) be corrupted by a sequence of "measurement" noise which is Gaussian with zero mean and variance $\sigma^2 = 0.1$. Repeat parts (c) and (d) for the noise-corrupted signal.



7

Implementation of Discrete-Time Systems

The focus of this chapter is on the realization of linear time-invariant discrete-time systems in either software or hardware. As we noted in Chapter 2, there are various configurations or structures for the realization of any FIR and IIR discrete-time system. In Chapter 2 we described the simplest of these structures, namely, the direct-form realizations. However, there are other more practical structures that offer some distinct advantages, especially when quantization effects are taken into consideration.

Of particular importance are the cascade, parallel, and lattice structures, which exhibit robustness in finite-word-length implementations. Also described in this chapter is the frequency-sampling realization for an FIR system, which often has the advantage of being computationally efficient when compared with alternative FIR realizations. Other important filter structures are obtained by employing a state-space formulation for linear time-invariant systems. An analysis of systems characterized by the state-variable form is presented in both the time and frequency domains.

In addition to describing the various structures for the realization of discrete-time systems, we also treat problems associated with quantization effects in the implementation of digital filters using finite-precision arithmetic. This treatment includes the effects on the filter frequency response characteristics resulting from coefficient quantization and the round-off noise effects inherent in the digital implementation of discrete-time systems.

7.1 STRUCTURES FOR THE REALIZATION OF DISCRETE-TIME SYSTEMS

Let us consider the important class of linear time-invariant discrete-time systems characterized by the general linear constant-coefficient difference equation

$$y(n) = -\sum_{k=1}^{N} a_k y(n-k) + \sum_{k=0}^{M} b_k x(n-k) \tag{7.1.1}$$

As we have shown by means of the z-transform, such a class of linear time-invariant discrete-time systems are also characterized by the rational system function

$$H(z) = \frac{\sum_{k=0}^{M} b_k z^{-k}}{1 + \sum_{k=1}^{N} a_k z^{-k}} \tag{7.1.2}$$

which is a ratio of two polynomials in z^{-1}. From the latter characterization, we obtain the zeros and poles of the system function, which depend on the choice of the system parameters $\{b_k\}$ and $\{a_k\}$ and which determine the frequency response characteristics of the system.

Our focus in this chapter is on the various methods of implementing (7.1.1) or (7.1.2) in either hardware, or in software on a programmable digital computer. We shall show that (7.1.1) or (7.1.2) can be implemented in a variety of ways depending on the form in which these two characterizations are arranged.

In general, we can view (7.1.1) as a computational procedure (an algorithm) for determining the output sequence $y(n)$ of the system from the input sequence $x(n)$. However, in various ways, the computations in (7.1.1) can be arranged into equivalent sets of difference equations. Each set of equations defines a computational procedure or an algorithm for implementing the system. From each set of equations we can construct a block diagram consisting of an interconnection of delay elements, multipliers, and adders. In Section 2.5 we referred to such a block diagram as a *realization* of the system or, equivalently, as a *structure* for realizing the system.

If the system is to be implemented in software, the block diagram or, equivalently, the set of equations that are obtained by rearranging (7.1.1), can be converted into a program that runs on a digital computer. Alternatively, the structure in block diagram form implies a hardware configuration for implementing the system.

Perhaps, the one issue that may not be clear to the reader at this point is why we are considering any rearrangements of (7.1.1) or (7.1.2). Why not just implement (7.1.1) or (7.1.2) directly without any rearrangement? If either (7.1.1) or (7.1.2) is rearranged in some manner, what are the benefits gained in the corresponding implementation?

These are the important questions which are answered in this chapter. At this point in our development, we simply state that the major factors that influence our choice of a specific realization are computational complexity, memory requirements, and finite-word-length effects in the computations.

Computational complexity refers to the number of arithmetic operations (multiplications, divisions, and additions) required to compute an output value $y(n)$ for the system. In the past, these were the only items used to measure computational complexity. However, with recent developments in the design and fabrication of rather sophisticated programmable digital signal processing chips, other factors,

such as the number of times a fetch from memory is performed or the number of times a comparison between two numbers is performed per output sample, have become important in assessing the computational complexity of a given realization of a system.

Memory requirements refers to the number of memory locations required to store the system parameters, past inputs, past outputs, and any intermediate computed values.

Finite-word-length effects or finite-precision effects refer to the quantization effects that are inherent in any digital implementation of the system, either in hardware or in software. The parameters of the system must necessarily be represented with finite precision. The computations that are performed in the process of computing an output from the system must be rounded- off or truncated to fit within the limited precision constraints of the computer or the hardware used in the implementation. Whether the computations are performed in fixed-point or floating-point arithmetic is another consideration. All these problems are usually called finite-word-length effects and are extremely important in influencing our choice of a system realization. We shall see that different structures of a system, which are equivalent for infinite precision, exhibit different behavior when finite-precision arithmetic is used in the implementation. Therefore, it is very important in practice to select a realization that is not very sensitive to finite-word-length effects.

Although these three factors are the major ones in influencing our choice of the realization of a system of the type described by either (7.1.1) or (7.1.2), other factors, such as whether the structure or the realization lends itself to parallel processing, or whether the computations can be pipelined, may play a role in our selection of the specific implementation. These additional factors are usually important in the realization of more complex digital signal processing algorithms.

In our discussion of alternative realizations, we concentrate on the three major factors just outlined. Occasionally, we will include some additional factors that may be important in some implementations.

7.2 STRUCTURES FOR FIR SYSTEMS

In general, an FIR system is described by the difference equation

$$y(n) = \sum_{k=0}^{M-1} b_k x(n-k) \tag{7.2.1}$$

or, equivalently, by the system function

$$H(z) = \sum_{k=0}^{M-1} b_k z^{-k} \tag{7.2.2}$$

Furthermore, the unit sample response of the FIR system is identical to the coef-

ficients $\{b_k\}$, that is,

$$h(n) = \begin{cases} b_n, & 0 \le n \le M-1 \\ 0, & \text{otherwise} \end{cases} \tag{7.2.3}$$

The length of the FIR filter is selected as M to conform with the established notation in the technical literature.

We shall present several methods for implementing an FIR system, beginning with the simplest structure, called the direct form. A second structure is the cascade-form realization. The third structure that we shall describe is the frequency-sampling realization. Finally, we present a lattice realization of an FIR system. In this discussion we follow the convention often used in the technical literature, which is to use $\{h(n)\}$ for the parameters of an FIR system.

In addition to the four realizations indicated above, an FIR system can be realized by means of the DFT, as described in Section 6.2. From one point of view, the DFT can be considered as a computational procedure rather than a structure for an FIR system. However, when the computational procedure is implemented in hardware, there is a corresponding structure for the FIR system. In practice, hardware implementations of the DFT are based on the use of the fast Fourier transform (FFT) algorithms described in Chapter 6.

7.2.1 Direct-Form Structure

The direct-form realization follows immediately from the nonrecursive difference equation given by (7.2.1) or, equivalently, by the convolution summation

$$y(n) = \sum_{k=0}^{M-1} h(k)x(n-k) \tag{7.2.4}$$

The structure is illustrated in Fig. 7.1.

We observe that this structure requires $M-1$ memory locations for storing the $M-1$ previous inputs, and has a complexity of M multiplications and $M-1$ additions per output point. Since the output consists of a weighted linear combination of $M-1$ past values of the input and the weighted current value of the input, the structure in Fig. 7.1, resembles a tapped delay line or a transversal

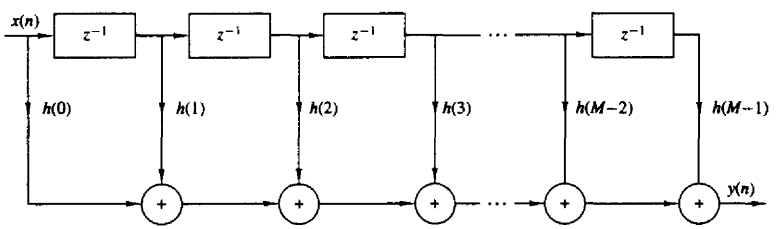


Figure 7.1 Direct-form realization of FIR system.

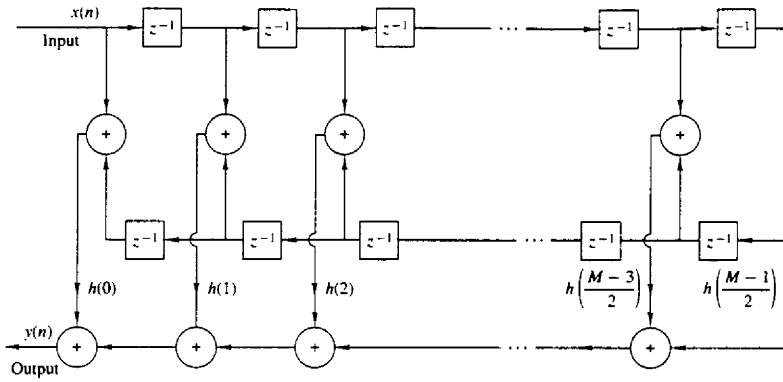


Figure 7.2 Direct-form realization of linear-phase FIR system (M odd).

system. Consequently, the direct-form realization is often called a transversal or tapped-delay-line filter.

When the FIR system has linear phase, as described in Section 8.2, the unit sample response of the system satisfies either the symmetry or asymmetry condition

$$h(n) = \pm h(M - 1 - n) \tag{7.2.5}$$

For such a system the number of multiplications is reduced from M to $M/2$ for M even and to $(M-1)/2$ for M odd. For example, the structure that takes advantage of this symmetry is illustrated in Fig. 7.2 for the case in which M is odd.

7.2.2 Cascade-Form Structures

The cascade realization follows naturally from the system function given by (7.2.2). It is a simple matter to factor $H(z)$ into second-order FIR systems so that

$$H(z) = \prod_{k=1}^{K} H_k(z) \tag{7.2.6}$$

where

$$H_k(z) = b_{k0} + b_{k1}z^{-1} + b_{k2}z^{-2} \qquad k = 1, 2, \ldots, K \tag{7.2.7}$$

and K is the integer part of $(M+1)/2$. The filter parameter b_0 may be equally distributed among the K filter sections, such that $b_0 = b_{10}b_{20}\cdots b_{K0}$ or it may be assigned to a single filter section. The zeros of $H(z)$ are grouped in pairs to produce the second-order FIR systems of the form (7.2.7). It is always desirable to form pairs of complex-conjugate roots so that the coefficients $\{b_{ki}\}$ in (7.2.7) are real valued. On the other hand, real-valued roots can be paired in any arbitrary manner. The cascade-form realization along with the basic second-order section are shown in Fig. 7.3.

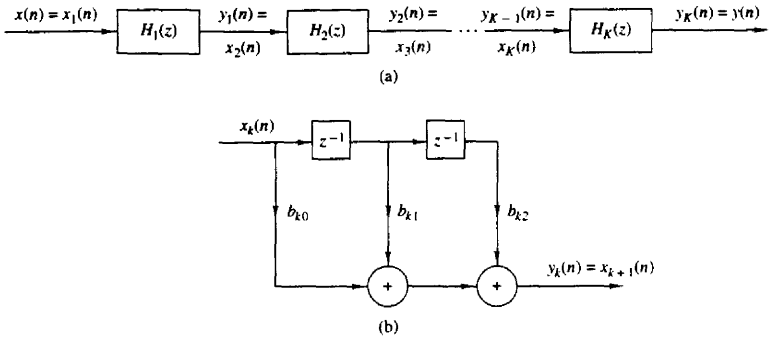


Figure 7.3 Cascade realization of an FIR system.

In the case of linear-phase FIR filters, the symmetry in $h(n)$ implies that the zeros of $H(z)$ also exhibit a form of symmetry. In particular, if z_k and z_k^* are a pair of complex-conjugate zeros then $1/z_k$ and $1/z_k^*$ are also a pair of complex-conjugate zeros (see Sec. 8.2). Consequently, we gain some simplification by forming fourth-order sections of the FIR system as follows

$$\begin{aligned} H_k(z) &= c_{k0}(1 - z_k z^{-1})(1 - z_k^* z^{-1})(1 - z^{-1}/z_k)(1 - z^{-1}/z_k^*) \\ &= c_{k0} + c_{k1}z^{-1} + c_{k2}z^{-2} + c_{k1}z^{-3} + z^{-4} \end{aligned} \tag{7.2.8}$$

where the coefficients $\{c_{k1}\}$ and $\{c_{k2}\}$ are functions of z_k. Thus, by combining the two pairs of poles to form a fourth-order filter section, we have reduced the number of multiplications from six to three (i.e., by a factor of 50%). Figure 7.4 illustrates the basic fourth-order FIR filter structure.

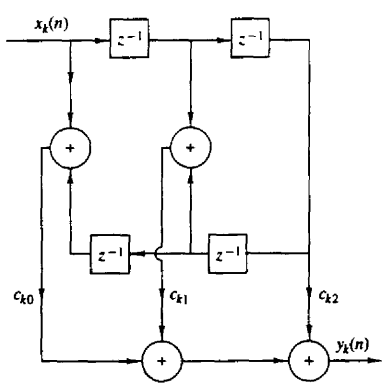


Figure 7.4 Fourth-order section in a cascade realization of an FIR system.

7.2.3 Frequency-Sampling Structures†

The frequency-sampling realization is an alternative structure for an FIR filter in which the parameters that characterize the filter are the values of the desired frequency response instead of the impulse response $h(n)$. To derive the frequency-sampling structure, we specify the desired frequency response at a set of equally spaced frequencies, namely

$$\omega_k = \frac{2\pi}{M}(k+\alpha) \qquad \begin{aligned} k &= 0, 1, \ldots, \frac{M-1}{2} \quad M \text{ odd} \\ k &= 0, 1, \ldots, \frac{M}{2} - 1 \quad M \text{ even} \\ \alpha &= 0 \text{ or } \tfrac{1}{2} \end{aligned}$$

and solve for the unit sample response $h(n)$ from these equally spaced frequency specifications. Thus we can write the frequency response as

$$H(\omega) = \sum_{n=0}^{M-1} h(n)e^{-j\omega n}$$

and the values of $H(\omega)$ at frequencies $\omega_k = (2\pi/M)(k+\alpha)$ are simply

$$\begin{aligned} H(k+\alpha) &= H\left(\frac{2\pi}{M}(k+\alpha)\right) \\ &= \sum_{n=0}^{M-1} h(n)e^{-j2\pi(k+\alpha)n/M} \qquad k = 0, 1, \ldots, M-1 \end{aligned} \tag{7.2.9}$$

The set of values $\{H(k+\alpha)\}$ are called the frequency samples of $H(\omega)$. In the case where $\alpha = 0$, $\{H(k)\}$ corresponds to the M-point DFT of $\{h(n)\}$.

It is a simple matter to invert (7.2.9) and express $h(n)$ in terms of the frequency samples. The result is

$$h(n) = \frac{1}{M}\sum_{k=0}^{M-1} H(k+\alpha)e^{j2\pi(k+\alpha)n/M} \qquad n = 0, 1, \ldots, M-1 \tag{7.2.10}$$

When $\alpha = 0$, (7.2.10) is simply the IDFT of $\{H(k)\}$. Now if we use (7.2.10) to substitute for $h(n)$ in the z-transform $H(z)$, we have

$$\begin{aligned} H(z) &= \sum_{n=0}^{M-1} h(n)z^{-n} \\ &= \sum_{n=0}^{M-1}\left[\frac{1}{M}\sum_{k=0}^{M-1} H(k+\alpha)e^{j2\pi(k+\alpha)n/M}\right]z^{-n} \end{aligned} \tag{7.2.11}$$

†The reader may also refer to Section 8.2.3 for additional discussion of frequency-sampling FIR filters.

By interchanging the order of the two summations in (7.2.11) and performing the summation over the index n we obtain

$$\begin{aligned} H(z) &= \sum_{k=0}^{M-1} H(k+\alpha)\left[\frac{1}{M}\sum_{n=0}^{M-1}(e^{j2\pi(k+\alpha)/M}z^{-1})^n\right] \\ &= \frac{1-z^{-M}e^{j2\pi\alpha}}{M}\sum_{k=0}^{M-1}\frac{H(k+\alpha)}{1-e^{j2\pi(k+\alpha)/M}z^{-1}} \end{aligned} \tag{7.2.12}$$

Thus the system function $H(z)$ is characterized by the set of frequency samples $\{H(k+\alpha)\}$ instead of $\{h(n)\}$.

We view this FIR filter realization as a cascade of two filters [i.e., $H(z) = H_1(z)H_2(z)$]. One is an all-zero filter, or a comb filter, with system function

$$H_1(z) = \frac{1}{M}(1 - z^{-M}e^{j2\pi\alpha}) \tag{7.2.13}$$

Its zeros are located at equally spaced points on the unit circle at

$$z_k = e^{j2\pi(k+\alpha)/M} \qquad k = 0, 1, \ldots, M-1$$

The second filter with system function

$$H_2(z) = \sum_{k=0}^{M-1}\frac{H(k+\alpha)}{1-e^{j2\pi(k+\alpha)/M}z^{-1}} \tag{7.2.14}$$

consists of a parallel bank of single-pole filters with resonant frequencies

$$p_k = e^{j2\pi(k+\alpha)/M} \qquad k = 0, 1, \ldots, M-1$$

Note that the pole locations are identical to the zero locations and that both occur at $\omega_k = 2\pi(k+\alpha)/M$, which are the frequencies at which the desired frequency response is specified. The gains of the parallel bank of resonant filters are simply the complex-valued parameters $\{H(k+\alpha)\}$. This cascade realization is illustrated in Fig. 7.5.

When the desired frequency response characteristic of the FIR filter is narrowband, most of the gain parameters $\{H(k+\alpha)\}$ are zero. Consequently, the corresponding resonant filters can be eliminated and only the filters with nonzero gains need be retained. The net result is a filter that requires fewer computations (multiplications and additions) than the corresponding direct-form realization. Thus we obtain a more efficient realization.

The frequency-sampling filter structure can be simplified further by exploiting the symmetry in $H(k+\alpha)$, namely, $H(k) = H^*(M-k)$ for $\alpha = 0$ and

$$H\left(k+\tfrac{1}{2}\right) = H^*\left(M-k-\tfrac{1}{2}\right) \qquad \text{for } \alpha = \tfrac{1}{2}$$

These relations are easily deduced from (7.2.9). As a result of this symmetry, a pair of single-pole filters can be combined to form a single two-pole filter with

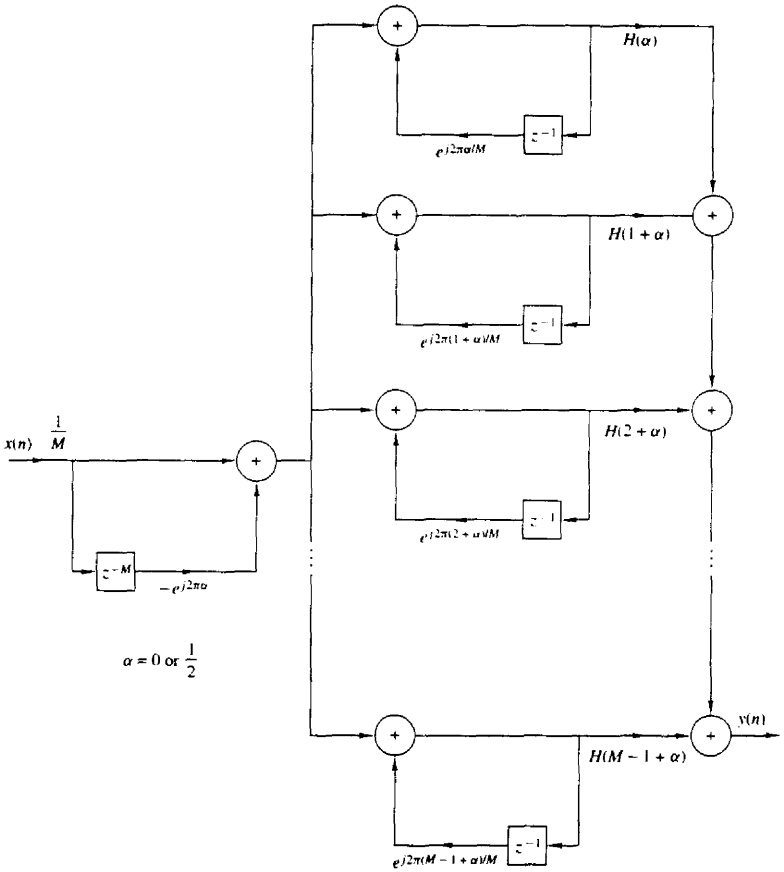


Figure 7.5 Frequency-sampling realization of FIR filter.

real-valued parameters. Thus for $\alpha = 0$ the system function $H_2(z)$ reduces to

$$H_2(z) = \frac{H(0)}{1 - z^{-1}} + \sum_{k=1}^{(M-1)/2} \frac{A(k) + B(k)z^{-1}}{1 - 2\cos(2\pi k/M)z^{-1} + z^{-2}} \qquad M \text{ odd}$$

$$H_2(z) = \frac{H(0)}{1 - z^{-1}} + \frac{H(M/2)}{1 + z^{-1}} + \sum_{k=1}^{(M/2)-1} \frac{A(k) + B(k)z^{-1}}{1 - 2\cos(2\pi k/M)z^{-1} + z^{-2}} \qquad M \text{ even} \tag{7.2.15}$$

where, by definition,

$$\begin{aligned} A(k) &= H(k) + H(M-k) \\ B(k) &= H(k)e^{-j2\pi k/M} + H(M-k)e^{j2\pi k/M} \end{aligned} \tag{7.2.16}$$

Similar expressions can be obtained for $\alpha = \frac{1}{2}$.

Example 7.2.1

Sketch the block diagram for the direct-form realization and the frequency-sampling realization of the $M = 32$, $\alpha = 0$, linear-phase (symmetric) FIR filter which has frequency samples

$$H\left(\frac{2\pi k}{32}\right) = \begin{cases} 1, & k = 0, 1, 2 \\ \frac{1}{2}, & k = 3 \\ 0, & k = 4, 5, \ldots, 15 \end{cases}$$

Compare the computational complexity of these two structures.

Solution Since the filter is symmetric, we exploit this symmetry and thus reduce the number of multiplications per output point by a factor of 2, from 32 to 16 in the direct-form realization. The number of additions per output point is 31. The block diagram of the direct realization is illustrated in Fig. 7.6.

We use the form in (7.2.13) and (7.2.15) for the frequency-sampling realization and drop all terms that have zero-gain coefficients {H(k)}. The nonzero coefficients are $H(k)$ and the corresponding pairs are $H(M-k)$, for $k = 0, 1, 2, 3$. The block diagram of the resulting realization is shown in Fig. 7.7. Since $H(0) = 1$, the single-pole filter requires no multiplication. The three double-pole filter sections require three multiplications each for a total of nine multiplications. The total number of additions is 13. Therefore, the frequency-sampling realization of this FIR filter is computationally more efficient than the direct-form realization.

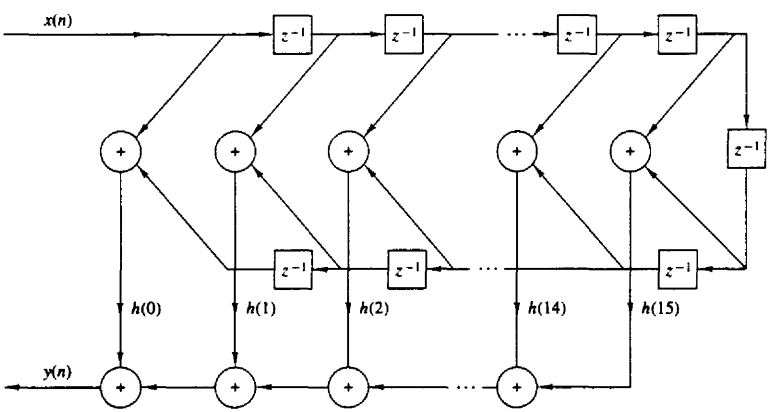


Figure 7.6 Direct-form realization of $M = 32$ FIR filter.

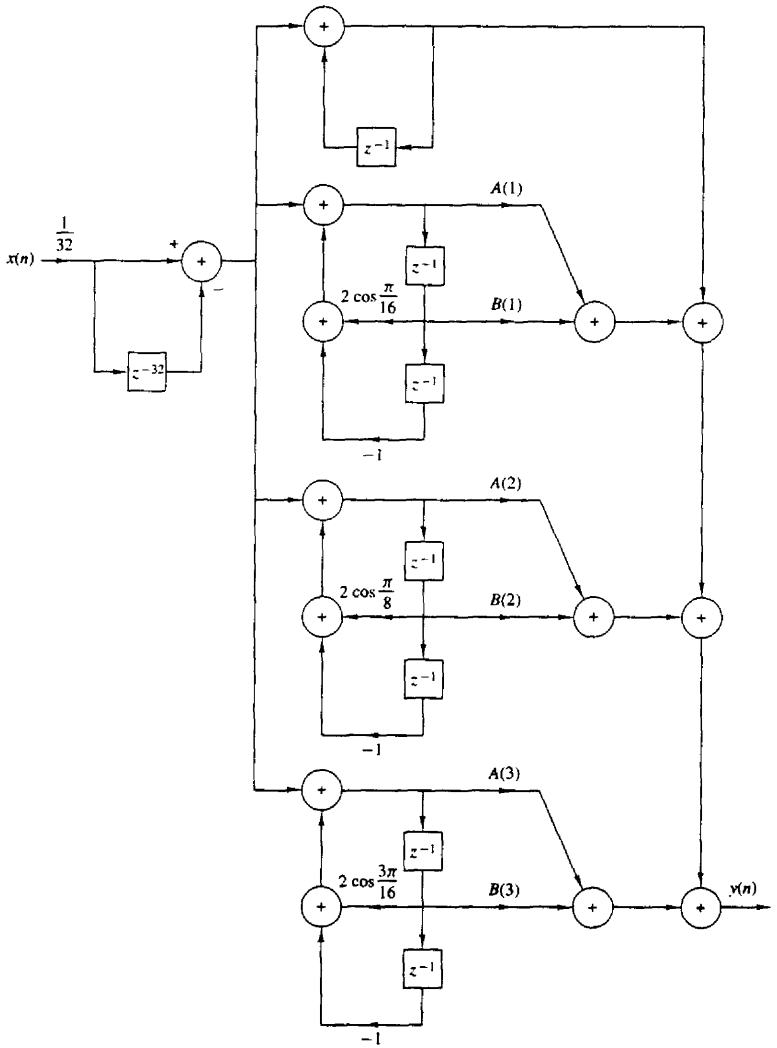


Figure 7.7 Frequency-sampling realization for the FIR filter in Example 7.2.1.

7.2.4 Lattice Structure

In this section we introduce another FIR filter structure, called the lattice filter or lattice realization. Lattice filters are used extensively in digital speech processing and in the implementation of adaptive filters.

Let us begin the development by considering a sequence of FIR filters with system functions

$$H_m(z) = A_m(z) \qquad m = 0, 1, 2, \ldots, M-1 \tag{7.2.17}$$

where, by definition, $A_m(z)$ is the polynomial

$$A_m(z) = 1 + \sum_{k=1}^{m} \alpha_m(k) z^{-k} \qquad m \geq 1 \tag{7.2.18}$$

and $A_0(z) = 1$. The unit sample response of the mth filter is $h_m(0) = 1$ and $h_m(k) = \alpha_m(k)$, $k = 1, 2, \ldots, m$. The subscript m on the polynomial $A_m(z)$ denotes the degree of the polynomial. For mathematical convenience, we define $\alpha_m(0) = 1$.

If $\{x(n)\}$ is the input sequence to the filter $A_m(z)$ and $\{y(n)\}$ is the output sequence, we have

$$y(n) = x(n) + \sum_{k=1}^{m} \alpha_m(k) x(n-k) \tag{7.2.19}$$

Two direct-form structures of the FIR filter are illustrated in Fig. 7.8.

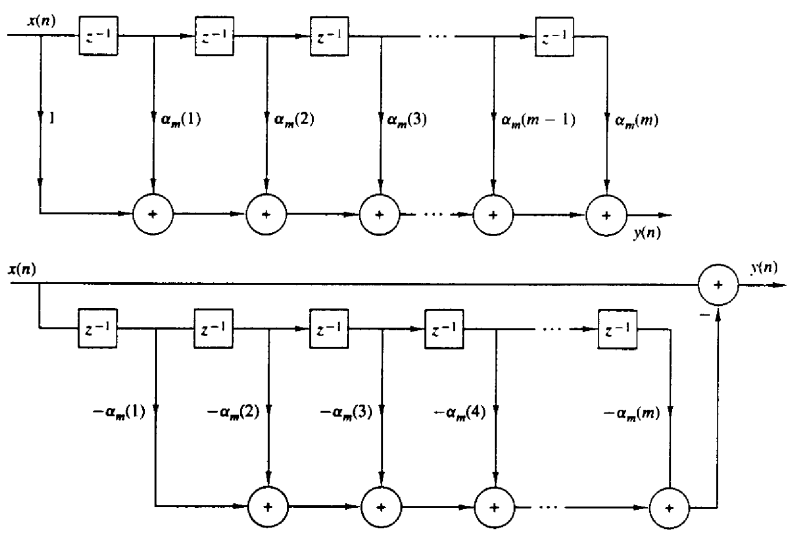


Figure 7.8 Direct-form realization of the FIR prediction filter.

In Chapter 11, we show that the FIR structures shown in Fig. 7.8 are intimately related with the topic of linear prediction, where

$$\hat{x}(n) = -\sum_{k=1}^{m} \alpha_m(k)x(n-k) \tag{7.2.20}$$

is the one-step forward predicted value of $x(n)$, based on m past inputs, and $y(n) = x(n) - \hat{x}(n)$, given by (7.2.19), represents the prediction error sequence. In this context, the top filter structure in Fig. 7.8 is called a *prediction error filter*.

Now suppose that we have a filter of order $m = 1$. The output of such a filter is

$$y(n) = x(n) + \alpha_1(1)x(n-1) \tag{7.2.21}$$

This output can also be obtained from a first-order or single-stage lattice filter, illustrated in Fig. 7.9, by exciting both of the inputs by $x(n)$ and selecting the output from the top branch. Thus the output is exactly (7.2.21), if we select $K_1 = \alpha_1(1)$. The parameter K_1 in the lattice is called a reflection coefficient and it is identical to the *reflection coefficient* introduced in the Schür–Cohn stability test described in Section 3.6.7.

Next, let us consider an FIR filter for which $m = 2$. In this case the output from a direct-form structure is

$$y(n) = x(n) + \alpha_2(1)x(n-1) + \alpha_2(2)x(n-2) \tag{7.2.22}$$

By cascading two lattice stages as shown in Fig. 7.10, it is possible to obtain the same output as (7.2.22). Indeed, the output from the first stage is

$$\begin{aligned} f_1(n) &= x(n) + K_1x(n-1) \\ g_1(n) &= K_1x(n) + x(n-1) \end{aligned} \tag{7.2.23}$$

The output from the second stage is

$$\begin{aligned} f_2(n) &= f_1(n) + K_2g_1(n-1) \\ g_2(n) &= K_2f_1(n) + g_1(n-1) \end{aligned} \tag{7.2.24}$$

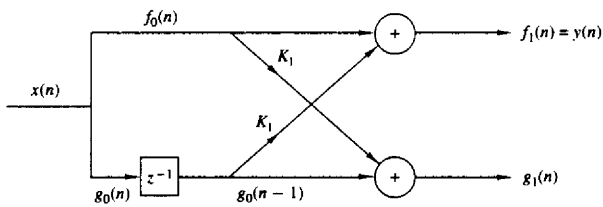


Figure 7.9 Single-stage lattice filter.

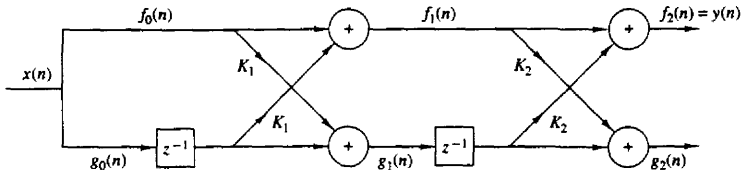


Figure 7.10 Two-stage lattice filter.

If we focus our attention on $f_2(n)$ and substitute for $f_1(n)$ and $g_1(n-1)$ from (7.2.23) into (7.2.24), we obtain

$$\begin{aligned} f_2(n) &= x(n) + K_1x(n-1) + K_2[K_1x(n-1) + x(n-2)] \\ &= x(n) + K_1(1+K_2)x(n-1) + K_2x(n-2) \end{aligned} \tag{7.2.25}$$

Now (7.2.25) is identical to the output of the direct-form FIR filter as given by (7.2.22), if we equate the coefficients, that is,

$$\alpha_2(2) = K_2 \qquad \alpha_2(1) = K_1(1+K_2) \tag{7.2.26}$$

or, equivalently,

$$K_2 = \alpha_2(2) \qquad K_1 = \frac{\alpha_2(1)}{1+\alpha_2(2)} \tag{7.2.27}$$

Thus the reflection coefficients K_1 and K_2 of the lattice can be obtained from the coefficients $\{\alpha_m(k)\}$ of the direct-form realization.

By continuing this process, one can easily demonstrate by induction, the equivalence between an mth-order direct-form FIR filter and an m-order or m-stage lattice filter. The lattice filter is generally described by the following set of order-recursive equations:

$$f_0(n) = g_0(n) = x(n) \tag{7.2.28}$$

$$f_m(n) = f_{m-1}(n) + K_m g_{m-1}(n-1) \qquad m = 1, 2, \ldots, M-1 \tag{7.2.29}$$

$$g_m(n) = K_m f_{m-1}(n) + g_{m-1}(n-1) \qquad m = 1, 2, \ldots, M-1 \tag{7.2.30}$$

Then the output of the $(M-1)$-stage filter corresponds to the output of an $(M-1)$-order FIR filter, that is,

$$y(n) = f_{M-1}(n)$$

Figure 7.11 illustrates an $(M-1)$-stage lattice filter in block diagram form along with a typical stage that shows the computations specified by (7.2.29) and (7.2.30).

As a consequence of the equivalence between an FIR filter and a lattice filter, the output $f_m(n)$ of an m-stage lattice filter can be expressed as

$$f_m(n) = \sum_{k=0}^{m} \alpha_m(k)x(n-k) \qquad \alpha_m(0) = 1 \tag{7.2.31}$$

Since (7.2.31) is a convolution sum, it follows that the z-transform relationship is

$$F_m(z) = A_m(z)X(z)$$

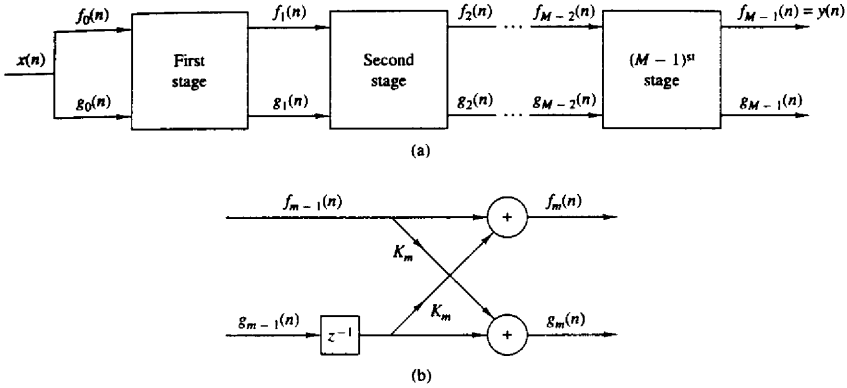


Figure 7.11 $(M-1)$-stage lattice filter.

or, equivalently,

$$A_m(z) = \frac{F_m(z)}{X(z)} = \frac{F_m(z)}{F_0(z)} \tag{7.2.32}$$

The other output component from the lattice, namely, $g_m(n)$, can also be expressed in the form of a convolution sum as in (7.2.31), by using another set of coefficients, say $\{\beta_m(k)\}$. That this in fact is the case, becomes apparent from observation of (7.2.23) and (7.2.24). From (7.2.23) we note that the filter coefficients for the lattice filter that produces $f_1(n)$ are $\{1, K_1\} = \{1, \alpha_1(1)\}$ while the coefficients for the filter with output $g_1(n)$ are $\{K_1, 1\} = \{\alpha_1(1), 1\}$. We note that these two sets of coefficients are in reverse order. If we consider the two-stage lattice filter, with the output given by (7.2.24), we find that $g_2(n)$ can be expressed in the form

$$\begin{aligned} g_2(n) &= K_2 f_1(n) + g_1(n-1) \\ &= K_2[x(n) + K_1 x(n-1)] + K_1 x(n-1) + x(n-2) \\ &= K_2 x(n) + K_1(1 + K_2)x(n-1) + x(n-2) \\ &= \alpha_2(2)x(n) + \alpha_2(1)x(n-1) + x(n-2) \end{aligned}$$

Consequently, the filter coefficients are $\{\alpha_2(2), \alpha_2(1), 1\}$, whereas the coefficients for the filter that produces the output $f_2(n)$ are $\{1, \alpha_2(1), \alpha_2(2)\}$. Here, again, the two sets of filter coefficients are in reverse order.

From this development it follows that the output $g_m(n)$ from an m-stage lattice filter can be expressed by the convolution sum of the form

$$g_m(n) = \sum_{k=0}^{m} \beta_m(k)x(n-k) \tag{7.2.33}$$

where the filter coefficients $\{\beta_m(k)\}$ are associated with a filter that produces $f_m(n) = y(n)$ but operates in reverse order. Consequently,

$$\beta_m(k) = \alpha_m(m-k) \qquad k = 0, 1, \ldots, m \tag{7.2.34}$$

with $\beta_m(m) = 1$.

In the context of linear prediction, suppose that the data $x(n)$, $x(n-1)$, ..., $x(n-m+1)$ is used to linearly predict the signal value $x(n-m)$ by use of a linear filter with coefficients $\{-\beta_m(k)\}$. Thus the predicted value is

$$\hat{x}(n-m) = -\sum_{k=0}^{m-1} \beta_m(k)x(n-k) \tag{7.2.35}$$

Since the data are run in reverse order through the predictor, the prediction performed in (7.2.35) is called *backward prediction*. In contrast, the FIR filter with system function $A_m(z)$ is called a *forward predictor*.

In the z-transform domain, (7.2.33) becomes

$$G_m(z) = B_m(z)X(z) \tag{7.2.36}$$

or, equivalently,

$$B_m(z) = \frac{G_m(z)}{X(z)} \tag{7.2.37}$$

where $B_m(z)$ represents the system function of the FIR filter with coefficients $\{\beta_m(k)\}$, that is,

$$B_m(z) = \sum_{k=0}^{m} \beta_m(k)z^{-k} \tag{7.2.38}$$

Since $\beta_m(k) = \alpha_m(m-k)$, (7.2.38) may be expressed as

$$\begin{aligned} B_m(z) &= \sum_{k=0}^{m} \alpha_m(m-k)z^{-k} \\ &= \sum_{l=0}^{m} \alpha_m(l)z^{l-m} \\ &= z^{-m}\sum_{l=0}^{m} \alpha_m(l)z^{l} \\ &= z^{-m}A_m(z^{-1}) \end{aligned} \tag{7.2.39}$$

The relationship in (7.2.39) implies that the zeros of the FIR filter with system function $B_m(z)$ are simply the reciprocals of the zeros of $A_m(z)$. Hence $B_m(z)$ is called the reciprocal or *reverse* polynomial of $A_m(z)$.

Now that we have established these interesting relationships between the direct-form FIR filter and the lattice structure, let us return to the recursive lattice equations in (7.2.28) through (7.2.30) and transfer them to the z-domain. Thus

we have

$$F_0(z) = G_0(z) = X(z) \tag{7.2.40}$$

$$F_m(z) = F_{m-1}(z) + K_m z^{-1} G_{m-1}(z) \qquad m = 1, 2, \ldots, M-1 \tag{7.2.41}$$

$$G_m(z) = K_m F_{m-1}(z) + z^{-1} G_{m-1}(z) \qquad m = 1, 2, \ldots, M-1 \tag{7.2.42}$$

If we divide each equation by $X(z)$, we obtain the desired results in the form

$$A_0(z) = B_0(z) = 1 \tag{7.2.43}$$

$$A_m(z) = A_{m-1}(z) + K_m z^{-1} B_{m-1}(z) \qquad m = 1, 2, \ldots, M-1 \tag{7.2.44}$$

$$B_m(z) = K_m A_{m-1}(z) + z^{-1} B_{m-1}(z) \qquad m = 1, 2, \ldots, M-1 \tag{7.2.45}$$

Thus a lattice stage is described in the z-domain by the matrix equation

$$\begin{bmatrix} A_m(z) \\ B_m(z) \end{bmatrix} = \begin{bmatrix} 1 & K_m \\ K_m & 1 \end{bmatrix} \begin{bmatrix} A_{m-1}(z) \\ z^{-1} B_{m-1}(z) \end{bmatrix} \tag{7.2.46}$$

Before concluding this discussion, it is desirable to develop the relationships for converting the lattice parameters $\{K_i\}$, that is, the reflection coefficients, to the direct-form filter coefficients $\{\alpha_m(k)\}$, and vice versa.

Conversion of lattice coefficients to direct-form filter coefficients. The direct-form FIR filter coefficients $\{\alpha_m(k)\}$ can be obtained from the lattice coefficients $\{K_i\}$ by using the following relations:

$$A_0(z) = B_0(z) = 1 \tag{7.2.47}$$

$$A_m(z) = A_{m-1}(z) + K_m z^{-1} B_{m-1}(z) \qquad m = 1, 2, \ldots, M-1 \tag{7.2.48}$$

$$B_m(z) = z^{-m} A_m(z^{-1}) \qquad m = 1, 2, \ldots, M-1 \tag{7.2.49}$$

The solution is obtained recursively, beginning with $m = 1$. Thus we obtain a sequence of $(M-1)$ FIR filters, one for each value of m. The procedure is best illustrated by means of an example.

Example 7.2.2

Given a three-stage lattice filter with coefficients $K_1 = \frac{1}{4}$, $K_2 = \frac{1}{2}$, $K_3 = \frac{1}{3}$, determine the FIR filter coefficients for the direct-form structure.

Solution We solve the problem recursively, beginning with (7.2.48) for $m = 1$. Thus we have

$$\begin{aligned} A_1(z) &= A_0(z) + K_1 z^{-1} B_0(z) \\ &= 1 + K_1 z^{-1} = 1 + \tfrac{1}{4} z^{-1} \end{aligned}$$

Hence the coefficients of an FIR filter corresponding to the single-stage lattice are $\alpha_1(0) = 1$, $\alpha_1(1) = K_1 = \frac{1}{4}$. Since $B_m(z)$ is the reverse polynomial of $A_m(z)$, we have

$$B_1(z) = \tfrac{1}{4} + z^{-1}$$

Next we add the second stage to the lattice. For $m = 2$, (7.2.48) yields

$$\begin{aligned} A_2(z) &= A_1(z) + K_2 z^{-1} B_1(z) \\ &= 1 + \tfrac{3}{8} z^{-1} + \tfrac{1}{2} z^{-2} \end{aligned}$$

Hence the FIR filter parameters corresponding to the two-stage lattice are $\alpha_2(0) = 1$, $\alpha_2(1) = \frac{3}{8}$, $\alpha_2(2) = \frac{1}{2}$. Also,

$$B_2(z) = \tfrac{1}{2} + \tfrac{3}{8} z^{-1} + z^{-2}$$

Finally, the addition of the third stage to the lattice results in the polynomial

$$\begin{aligned} A_3(z) &= A_2(z) + K_3 z^{-1} B_2(z) \\ &= 1 + \tfrac{13}{24} z^{-1} + \tfrac{5}{8} z^{-2} + \tfrac{1}{3} z^{-3} \end{aligned}$$

Consequently, the desired direct-form FIR filter is characterized by the coefficients

$$\alpha_3(0) = 1 \qquad \alpha_3(1) = \tfrac{13}{24} \qquad \alpha_3(2) = \tfrac{5}{8} \qquad \alpha_3(3) = \tfrac{1}{3}$$

As this example illustrates, the lattice structure with parameters $K_1, K_2, \ldots, K_m$, corresponds to a class of m direct-form FIR filters with system functions $A_1(z), A_2(z), \ldots, A_m(z)$. It is interesting to note that a characterization of this class of m FIR filters in direct form requires $m(m+1)/2$ filter coefficients. In contrast, the lattice-form characterization requires only the m reflection coefficients $\{K_i\}$. The reason that the lattice provides a more compact representation for the class of m FIR filters is simply due to the fact that the addition of stages to the lattice does not alter the parameters of the previous stages. On the other hand, the addition of the mth stage to a lattice with $(m-1)$ stages results in a FIR filter with system function $A_m(z)$ that has coefficients totally different from the coefficients of the lower-order FIR filter with system function $A_{m-1}(z)$.

A formula for determining the filter coefficients $\{\alpha_m(k)\}$ recursively can be easily derived from polynomial relationships in (7.2.47) through (7.2.49). From the relationship in (7.2.48) we have

$$\begin{aligned} A_m(z) &= A_{m-1}(z) + K_m z^{-1} B_{m-1}(z) \\ \sum_{k=0}^{m} \alpha_m(k) z^{-k} &= \sum_{k=0}^{m-1} \alpha_{m-1}(k) z^{-k} + K_m \sum_{k=0}^{m-1} \alpha_{m-1}(m-1-k) z^{-(k+1)} \end{aligned} \tag{7.2.50}$$

By equating the coefficients of equal powers of z^{-1} and recalling that $\alpha_m(0) = 1$ for $m = 1, 2, \ldots, M-1$, we obtain the desired recursive equation for the FIR filter coefficients in the form

$$\alpha_m(0) = 1 \tag{7.2.51}$$

$$\alpha_m(m) = K_m \tag{7.2.52}$$

$$\begin{aligned} \alpha_m(k) &= \alpha_{m-1}(k) + K_m \alpha_{m-1}(m-k) \\ &= \alpha_{m-1}(k) + \alpha_m(m) \alpha_{m-1}(m-k) \end{aligned} \qquad \begin{matrix} 1 \le k \le m-1 \\ m = 1, 2, \ldots, M-1 \end{matrix} \tag{7.2.53}$$

We note that (7.2.51) through (7.2.53) are simply the Levinson–Durbin recursive equations given in Chapter 11.

Conversion of direct-form FIR filter coefficients to lattice coefficients. Suppose that we are given the FIR coefficients for the direct-form realization or, equivalently, the polynomial $A_m(z)$, and we wish to determine the corresponding lattice filter parameters $\{K_i\}$. For the m-stage lattice we immediately obtain the parameter $K_m = \alpha_m(m)$. To obtain K_{m-1} we need the polynomials $A_{m-1}(z)$ since, in general, K_m is obtained from the polynomial $A_m(z)$ for $m = M-1, M-2, \ldots, 1$. Consequently, we need to compute the polynomials $A_m(z)$ starting from $m = M-1$ and "stepping down" successively to $m = 1$.

The desired recursive relation for the polynomials is easily determined from (7.2.44) and (7.2.45). We have

$$\begin{aligned} A_m(z) &= A_{m-1}(z) + K_m z^{-1} B_{m-1}(z) \\ &= A_{m-1}(z) + K_m[B_m(z) - K_m A_{m-1}(z)] \end{aligned}$$

If we solve for $A_{m-1}(z)$, we obtain

$$A_{m-1}(z) = \frac{A_m(z) - K_m B_m(z)}{1 - K_m^2} \qquad m = M-1, M-2, \ldots, 1 \tag{7.2.54}$$

which is just the step-down recursion used in the Schür–Cohn stability test described in Section 3.6.7. Thus we compute all lower-degree polynomials $A_m(z)$ beginning with $A_{M-1}(z)$ and obtain the desired lattice coefficients from the relation $K_m = \alpha_m(m)$. We observe that the procedure works as long as $|K_m| \neq 1$ for $m = 1, 2, \ldots, M-1$.

Example 7.2.3

Determine the lattice coefficients corresponding to the FIR filter with system function

$$H(z) = A_3(z) = 1 + \tfrac{13}{24}z^{-1} + \tfrac{5}{8}z^{-2} + \tfrac{1}{3}z^{-3}$$

Solution First we note that $K_3 = \alpha_3(3) = \frac{1}{3}$. Furthermore,

$$B_3(z) = \tfrac{1}{3} + \tfrac{5}{8}z^{-1} + \tfrac{13}{24}z^{-2} + z^{-3}$$

The step-down relationship in (7.2.54) with $m = 3$ yields

$$\begin{aligned} A_2(z) &= \frac{A_3(z) - K_3 B_3(z)}{1 - K_3^2} \\ &= 1 + \tfrac{3}{8}z^{-1} + \tfrac{1}{2}z^{-2} \end{aligned}$$

Hence $K_2 = \alpha_2(2) = \frac{1}{2}$ and $B_2(z) = \frac{1}{2} + \frac{3}{8}z^{-1} + z^{-1}$. By repeating the step-down recursion in (7.2.51), we obtain

$$\begin{aligned} A_1(z) &= \frac{A_2(z) - K_2 B_2(z)}{1 - K_2^2} \\ &= 1 + \tfrac{1}{4}z^{-1} \end{aligned}$$

Hence $K_1 = \alpha_1(1) = \frac{1}{4}$.

From the step-down recursive equation in (7.2.54), it is relatively easy to obtain a formula for recursively computing K_m, beginning with $m = M - 1$ and stepping down to $m = 1$. For $m = M - 1, M - 2, \ldots, 1$ we have

$$K_m = \alpha_m(m) \quad \alpha_{m-1}(0) = 1 \tag{7.2.55}$$

$$\alpha_{m-1}(k) = \frac{\alpha_m(k) - K_m\beta_m(k)}{1 - K_m^2}$$

$$= \frac{\alpha_m(k) - \alpha_m(m)\alpha_m(m-k)}{1 - \alpha_m^2(m)} \qquad 1 \le k \le m-1 \tag{7.2.56}$$

which is again the recursion we introduced in the Schür–Cohn stability test.

As indicated above, the recursive equation in (7.2.56) breaks down if any lattice parameters $|K_m| = 1$. If this occurs, it is indicative of the fact that the polynomial $A_{m-1}(z)$ has a root on the unit circle. Such a root can be factored out from $A_{m-1}(z)$ and the iterative process in (7.2.56) is carried out for the reduced-order system.

7.3 STRUCTURES FOR IIR SYSTEMS

In this section we consider different IIR systems structures described by the difference equation in (7.1.1) or, equivalently, by the system function in (7.1.2). Just as in the case of FIR systems, there are several types of structures or realizations, including direct-form structures, cascade-form structures, lattice structures, and lattice-ladder structures. In addition, IIR systems lend themselves to a parallel-form realization. We begin by describing two direct-form realizations.

7.3.1 Direct-Form Structures

The rational system function as given by (7.1.2) that characterizes an IIR system can be viewed as two systems in cascade, that is,

$$H(z) = H_1(z)H_2(z) \tag{7.3.1}$$

where $H_1(z)$ consists of the zeros of $H(z)$, and $H_2(z)$ consists of the poles of $H(z)$,

$$H_1(z) = \sum_{k=0}^{M} b_k z^{-k} \tag{7.3.2}$$

and

$$H_2(z) = \frac{1}{1 + \sum_{k=1}^{N} a_k z^{-k}} \tag{7.3.3}$$

In Section 2.5.1 we describe two different direct-form realizations, characterized by whether $H_1(z)$ precedes $H_2(z)$, or vice versa. Since $H_1(z)$ is an FIR system, its direct-form realization was illustrated in Fig. 7.1. By attaching the all-pole

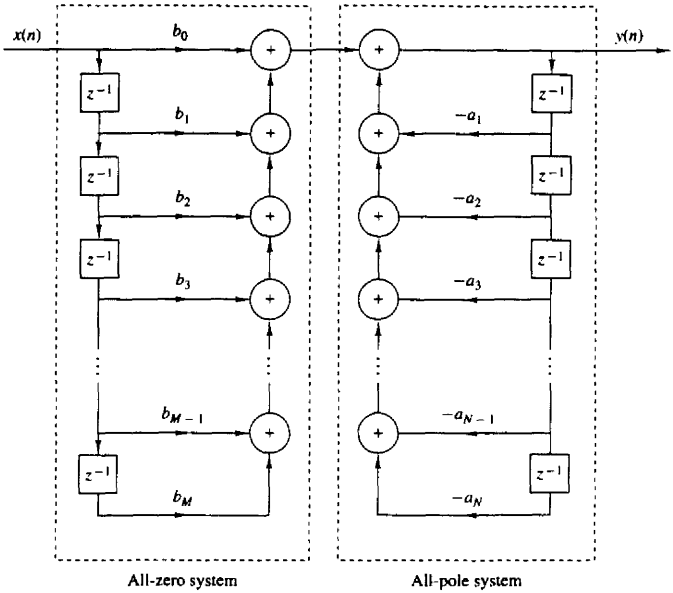


Figure 7.12 Direct form I realization.

system in cascade with $H_1(z)$, we obtain the direct form I realization depicted in Fig. 7.12. This realization requires $M + N + 1$ multiplications, $M + N$ additions, and $M + N + 1$ memory locations.

If the all-pole filter $H_2(z)$ is placed before the all-zero filter $H_1(z)$, a more compact structure is obtained as illustrated in Section 2.5.1. Recall that the difference equation for the all-pole filter is

$$w(n) = -\sum_{k=1}^{N} a_k w(n-k) + x(n) \tag{7.3.4}$$

Since $w(n)$ is the input to the all-zero system, its output is

$$y(n) = \sum_{k=0}^{M} b_k w(n-k) \tag{7.3.5}$$

We note that both (7.3.4) and (7.3.5) involve delayed versions of the sequence $\{w(n)\}$. Consequently, only a single delay line or a single set of memory locations is required for storing the past values of $\{w(n)\}$. The resulting structure that implements (7.3.4) and (7.3.5) is called a direct form II realization and is depicted in Fig. 7.13. This structure requires $M + N + 1$ multiplications, $M + N$ additions,

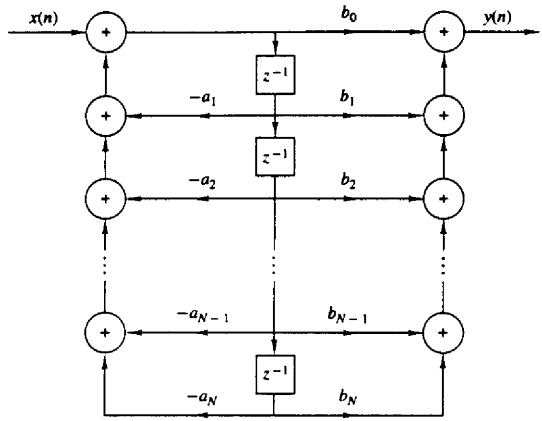


Figure 7.13 Direct form II realization ($N = M$).

and the maximum of $\{M, N\}$ memory locations. Since the direct form II realization minimizes the number of memory locations, it is said to be *canonic*. However, we should indicate that other IIR structures also possess this property, so that this terminology is perhaps unjustified.

The structures in Figs. 7.12 and 7.13 are both called "direct form" realizations because they are obtained directly from the system function $H(z)$ without any rearrangement of $H(z)$. Unfortunately, both structures are extremely sensitive to parameter quantization, in general, and are not recommended in practical applications. This topic is discussed in detail in Section 7.6, where we demonstrate that when N is large, a small change in a filter coefficient due to parameter quantization, results in a large change in the location of the poles and zeros of the system.

7.3.2 Signal Flow Graphs and Transposed Structures

A signal flow graph provides an alternative, but equivalent, graphical representation to a block diagram structure that we have been using to illustrate various system realizations. The basic elements of a flow graph are branches and nodes. A signal flow graph is basically a set of directed branches that connect at nodes. By definition, the signal out of a branch is equal to the branch gain (system function) times the signal into the branch. Furthermore, the signal at a node of a flow graph is equal to the sum of the signals from all branches connecting to the node.

To illustrate these basic notions, let us consider the two-pole and two-zero IIR system depicted in block diagram form in Fig. 7.14a. The system block

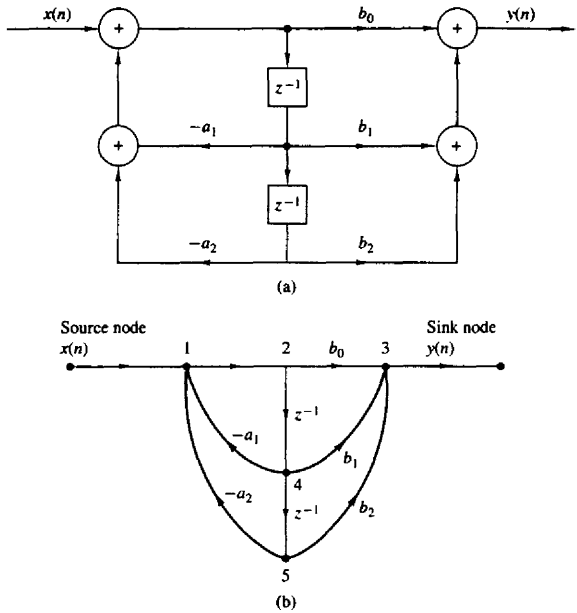


Figure 7.14 Second-order filter structure (a) and its signal flow graph (b).

diagram can be converted to the signal flow graph shown in Fig. 7.14b. We note that the flow graph contains five nodes labeled 1 through 5. Two of the nodes (1, 3) are summing nodes (i.e., they contain adders), while the other three nodes represent branching points. Branch transmittances are indicated for the branches in the flow graph. Note that a delay is indicated by the branch transmittance z^{-1}. When the branch transmittance is unity, it is left unlabeled. The input to the system originates at a *source node* and the output signal is extracted at a *sink node*.

We observe that the signal flow graph contains the same basic information as the block diagram realization of the system. The only apparent difference is that *both* branch points and adders in the block diagram are represented by nodes in the signal flow graph.

The subject of linear signal flow graphs is an important one in the treatment of networks and many interesting results are available. One basic notion involves the transformation of one flow graph into another without changing the basic input–output relationship. Specifically, one technique that is useful in deriving new system structures for FIR and IIR systems stems from the *transposition* or *flow-graph reversal theorem*. This theorem simply states that if we reverse the

directions of all branch transmittances and interchange the input and output in the flow graph, the system function remains unchanged. The resulting structure is called a *transposed structure* or a *transposed form*.

For example, the transposition of the signal flow graph in Fig. 7.14b is illustrated in Fig. 7.15a. The corresponding block diagram realization of the transposed form is depicted in Fig. 7.15b. It is interesting to note that the transposition of the original flow graph resulted in branching nodes becoming adder nodes, and vice versa. In Section 7.5 we provide a proof of the transposition theorem by using state-space techniques.

Let us apply the transposition theorem to the direct form II structure. First, we reverse all the signal flow directions in Fig. 7.13. Second, we change nodes into adders and adders into nodes, and finally, we interchange the input and the output. These operations result in the transposed direct form II structure shown in Fig. 7.16. This structure can be redrawn as in Fig. 7.17, which shows the input on the left and the output on the right.

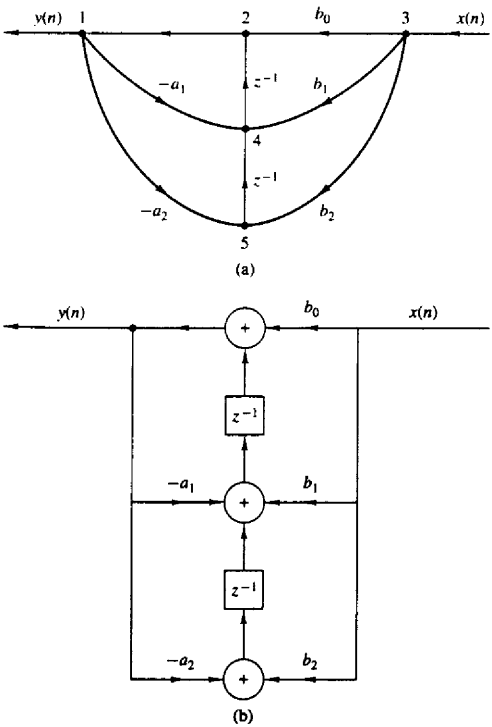


Figure 7.15 Signal flow graph of transposed structure (a) and its realization (b).